

JAVA OOP

INTERVIEW QUESTION BANK

Basic to Advanced | 200+ Questions | Scenario-based | Coding Problems

ABOUT THIS QUESTION BANK

This comprehensive question bank covers Java OOP from basic to advanced level, structured for Java Backend Developer interviews. Each question includes a perfect answer, what the interviewer is testing, follow-up questions, common mistakes to avoid, real-world analogies, and code examples.

Designed for: 6th-semester CSE students and developers preparing for entry-level to mid-level Java backend roles.

HOW TO USE

- Study each topic in sequence — basics build into advanced concepts.
- For every question, first write your own answer, then compare with the PERFECT ANSWER.
- Practice the speak-aloud sections in front of a mirror or record yourself.
- Mark questions with priority for last-minute revision.
- Solve all coding problems on paper before typing in IDE.

TABLE OF CONTENTS

Part	Section	Difficulty
PART 1	Basic Level — Classes, Objects, Constructors, this, static	☆☆ Easy
PART 2	Intermediate Level — Inheritance, Polymorphism, Encapsulation, Abstraction, Access Modifiers	☆☆☆ Medium
PART 3	Advanced Level — Interface, Abstract Class, Composition, Coupling/Cohesion	☆☆☆☆ Hard
PART 4	Scenario-based & Design Questions — Library, ATM, Vehicle, Employee systems	☆☆☆☆ Hard
PART 5	Coding Questions — OOP-focused implementation problems	☆☆☆☆ Hard
PART 6	Rapid Fire Round — 50 one-line answers	☆☆☆ Medium
PART 7	VVI Interview Topics — Priority list by frequency	☆☆☆☆☆ Critical

★ VVI: PRIORITY ROADMAP

Week 1: Master Part 1 (Basic). Build solid foundation.

Week 2: Part 2 (Intermediate). Inheritance + Polymorphism are non-negotiable.

Week 3: Part 3 (Advanced). Interface vs Abstract is asked in 90% interviews.

Week 4: Parts 4–5 (Design + Coding). Practice on paper first.

Day before interview: Parts 6–7 (Rapid Fire + VVI list).

PART 1: BASIC LEVEL (Fresher / Entry Level)

This section covers the absolute fundamentals of Java OOP. Every question here is asked in 99% of fresher interviews. Master these before moving to Part 2.

TOPIC 1.1: CLASSES & OBJECTS

Q1. What is a class and object? Explain with real-world example.

✓ PERFECT ANSWER

A class is a blueprint or template that defines properties (attributes) and behaviors (methods). An object is an instance of that class — a real entity created in memory using the blueprint.

Real-world example:

- Class = Car blueprint (defines: color, model, speed as properties; start(), stop() as methods).
- Object = Specific car like 'myCar' (actual car with color = Red, model = Honda).

```
class Car {  
    String color;  
    String model;  
    int speed;  
  
    void start() {  
        System.out.println("Car started");  
    }  
  
    void stop() {  
        System.out.println("Car stopped");  
    }  
}  
  
public class Demo {  
    public static void main(String[] args) {  
        Car myCar = new Car();        // Object creation  
        myCar.color = "Red";  
        myCar.model = "Honda";  
        myCar.start();                // Output: Car started  
    }  
}
```



WHAT INTERVIEWER TESTS

- Basic understanding of OOP fundamentals.
- Ability to relate abstract concepts to real-world entities.
- Clear distinction between blueprint and instance.
- Code explanation skills.

+ FOLLOW-UP QUESTIONS

- How many objects can be created from one class? → Unlimited (memory-permitting).
- What is the difference between class and object? → Class is a logical template; object is a physical instance occupying memory.
- Where are objects stored? → In heap memory; references are stored on the stack.
- Can a class exist without any object? → Yes — classes with only static members (e.g., Math).

⚠ COMMON MISTAKES

- ❌ Saying 'Class and object are the same thing'.
- ❌ Unable to give a concrete real-world example.
- ❌ Confusing class declaration with object instantiation.
- ✅ Always pair the concept with code AND a real-world analogy.

💡 HOW TO SPEAK ALOUD

"A class is like an architectural blueprint of a house — it defines what the house will look like but isn't a house itself. An object is the actual house built from that blueprint. We can build many houses (objects) from the same blueprint (class). In Java, we use the 'new' keyword to construct objects in heap memory."

Q2. What are the different ways to create an object in Java?

✓ PERFECT ANSWER

Java provides five primary ways to create objects:

1. Using the new keyword — most common.
2. Using `Class.forName().newInstance()` — reflection (deprecated since Java 9).
3. Using `clone()` method — creates a copy of an existing object.
4. Using deserialization — reading object from stream/file.
5. Using factory methods — e.g., `String.valueOf(10)`, `Integer.parseInt()`.

```
// 1. Using new keyword
Student s1 = new Student();

// 2. Using reflection
Student s2 = (Student) Class.forName("Student")
                        .getDeclaredConstructor()
                        .newInstance();

// 3. Using clone()
Student s3 = (Student) s1.clone();

// 4. Using deserialization
ObjectInputStream in = new ObjectInputStream(new FileInputStream("s.ser"));
Student s4 = (Student) in.readObject();

// 5. Using factory method
String str = String.valueOf(100);
```

+ FOLLOW-UP QUESTIONS

- Which is the most commonly used? → new keyword (95% of cases).
- Which one doesn't call a constructor? → clone() and deserialization.
- Where is the object stored in each case? → Always heap; only the reference location differs.

Q3. What is the difference between a Java class and a JavaBean?

✓ PERFECT ANSWER

A Java class is any class with fields and methods. A JavaBean is a Java class that follows strict conventions to be portable and reusable.

Feature	Regular Class	JavaBean
Serializable	Optional	Must implement Serializable

Feature	Regular Class	JavaBean
Constructor	Any kind allowed	Must have a public no-arg constructor
Fields	Any access modifier	Must be private
Access	Direct or via methods	Only via getter/setter methods
Naming	No convention required	getXxx() / setXxx() / isXxx()

```
import java.io.Serializable;

public class Employee implements Serializable {
    private int id;
    private String name;

    public Employee() {} // Required no-arg
    constructor

    public int getId() { return id; }
    public void setId(int id) { this.id = id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
}
```

Q4. Where are objects stored in memory? Stack vs Heap?

✓ PERFECT ANSWER

Java memory is divided into two main regions for object management:

Stack memory: Stores method calls, local variables, and object references. Fast access, fixed-size, follows LIFO order. Each thread has its own stack.

Heap memory: Stores all objects and instance variables. Shared across all threads. Larger but slower than stack. Managed by the Garbage Collector.

```

void method() {
    int x = 10;                // 'x' lives on stack
    Student s = new Student(); // 's' (reference) on stack
                                // 'new Student()' object on HEAP
}

// When method returns, stack frame is destroyed.
// The Student object becomes eligible for GC if no other reference exists.

```

Aspect	Stack	Heap
What is stored	References, primitives, method frames	Objects, instance variables
Access speed	Faster	Slower
Lifetime	Method scope	Until garbage collected
Thread safety	Per-thread (private)	Shared (needs sync)
Size	Fixed at thread start	Configurable, dynamic

Q5. What happens when you write `Car c = new Car();`?

✓ PERFECT ANSWER

This single statement triggers a five-step process:

6. Car class is loaded by ClassLoader (if not already loaded).
7. Memory is allocated on the heap for the new Car object.
8. All instance variables are initialized to default values (0, null, false).
9. The constructor `Car()` is invoked — instance initializer blocks and explicit initializations run.
10. The reference to the new heap object is assigned to the local variable `c` on the stack.



WHAT INTERVIEWER TESTS

This tests deep understanding of JVM internals — a strong signal for backend roles where memory management matters for performance.

Q6. Can a Java program run without creating any object?

✓ PERFECT ANSWER

Yes. A Java program can execute without explicit object creation if it relies only on static members and the main method, since main() is itself static and can be invoked without instantiation.

However, under the hood, the JVM creates several internal objects (Class object, ClassLoader, String pool entries, etc.). So 'no objects at all' is technically impossible, but no user-defined object creation is possible.

```
public class NoObjectDemo {  
    static int counter = 0;  
  
    static void increment() {  
        counter++;  
    }  
  
    public static void main(String[] args) {  
        increment();  
        increment();  
        System.out.println(counter);    // Output: 2 – no object created  
    }  
}
```

Q7. What is the difference between an instance variable and a local variable?

Feature	Instance Variable	Local Variable
Declared in	Inside class, outside methods	Inside a method/block/constructor
Memory	Heap (with object)	Stack (with method frame)
Default value	Yes (0, null, false)	No — must be initialized
Scope	Entire class via this	Only within declaring block
Access modifier	Allowed (private/public/etc.)	Not allowed
Lifetime	Until object is GC'd	Until method exits

```
class Demo {  
    int instanceVar;           // Instance: default 0, lives on heap  
  
    void method() {  
        int localVar;         // Local: no default, must initialize before  
use  
        // System.out.println(localVar); // Compile error: not initialized  
        localVar = 5;  
        System.out.println(localVar);    // OK: 5  
    }  
}
```

Q8. What is the difference between Object class and the keyword Object?

✓ PERFECT ANSWER

There is no 'Object keyword' in Java — only the `java.lang.Object` class. This class is the implicit superclass of every class in Java. If a class does not explicitly extend another, it implicitly extends `Object`.

Key methods inherited from Object: `equals()`, `hashCode()`, `toString()`, `getClass()`, `clone()`, `wait()`, `notify()`, `notifyAll()`, `finalize()`.

```
class Student { }    // Implicitly: class Student extends Object { }  
  
Student s = new Student();  
System.out.println(s.toString());    // Inherited from Object  
System.out.println(s.hashCode());    // Inherited from Object
```

Q9. What happens to an object when it goes out of scope?

✓ PERFECT ANSWER

When all references to an object are lost (out of scope, reassigned, or set to null), the object becomes eligible for garbage collection. The JVM's GC reclaims its heap memory at an unspecified later time.

Important: Going out of scope $\neq$ immediate destruction. GC runs when JVM decides — usually under memory pressure.

```
void method() {  
    Student s = new Student();    // Object created on heap  
    // ... use s  
}    // 's' reference removed from stack; object now eligible for GC  
    // Actual collection happens later when GC runs
```

Q10. What is an anonymous object? When is it useful?

✓ PERFECT ANSWER

An anonymous object is an object created without assigning it to a reference variable. It is used once and immediately becomes eligible for garbage collection.

Use cases:

- Calling a method only once on a newly-created object.
- Passing temporary objects as method arguments.
- Implementing one-shot listeners or callbacks.

```
class Calculator {  
    int square(int n) { return n * n; }  
}  
  
public class Demo {  
    public static void main(String[] args) {  
        // Anonymous object – used once  
        System.out.println(new Calculator().square(5));    // Output: 25  
        // After this line, the Calculator object is eligible for GC  
    }  
}
```

SUMMARY: Classes & Objects

- Class = blueprint (logical); Object = instance (physical).
- Objects live on heap; references live on stack.
- Every class implicitly extends java.lang.Object.
- Anonymous objects are useful for one-time operations.
- Garbage Collector reclaims memory when no references remain.

TOPIC 1.2: CONSTRUCTORS

Q1. What is a constructor? Types of constructors?

✓ PERFECT ANSWER

A constructor is a special method invoked automatically when an object is created, used to initialize the object's state. It has the same name as the class and no return type (not even void).

Three types in Java:

11. Default Constructor — no parameters, provided by compiler if none defined.
12. Parameterized Constructor — accepts arguments to initialize fields.
13. Copy Constructor — takes another object of the same class (not built-in; user-defined).

```
class Student {  
    String name;  
    int age;  
  
    // 1. Default constructor  
    Student() {  
        name = "Unknown";  
        age = 0;  
    }  
  
    // 2. Parameterized constructor  
    Student(String n, int a) {  
        name = n;  
        age = a;  
    }  
  
    // 3. Copy constructor  
    Student(Student s) {  
        name = s.name;  
        age = s.age;  
    }  
}  
  
Student s1 = new Student();           // Default  
Student s2 = new Student("Md", 21);  // Parameterized  
Student s3 = new Student(s2);         // Copy
```



WHAT INTERVIEWER TESTS

- Constructor concept clarity.
- Awareness of all three types.
- Understanding compiler-provided default constructor.

+ FOLLOW-UP QUESTIONS

- Can a constructor be private? → Yes, for Singleton pattern or static factory methods.
- What is constructor chaining? → Calling one constructor from another using `this()` or `super()`.
- Does constructor have a return type? → No, not even void.

Q2. What is the difference between a constructor and a method?

Feature	Constructor	Method
Purpose	Initialize object	Define behavior / perform task
Name	Same as class	Any valid identifier
Return type	None (not even void)	Required (void or type)
Invocation	Automatically on new	Explicitly by caller
Inheritance	Not inherited	Inherited
this/super call	Must be first statement	Can be anywhere
Modifiers	Cannot be static/final/abstract	Can be all of these

Q3. What is constructor chaining? Explain `this()` and `super()`.**✓ PERFECT ANSWER**

Constructor chaining is the process of calling one constructor from another, either within the same class (using `this()`) or from the parent class (using `super()`).

Rules:

- `this()/super()` must be the first statement in a constructor.
- You cannot use both `this()` and `super()` in the same constructor.
- If neither is written, the compiler implicitly inserts `super()`.

```
class Animal {  
    Animal() {  
        System.out.println("Animal constructor");  
    }  
    Animal(String type) {  
        System.out.println("Animal: " + type);  
    }  
}  
  
class Dog extends Animal {  
    Dog() {  
        this("Labrador");           // calls Dog(String)  
        System.out.println("Dog default");  
    }  
  
    Dog(String breed) {  
        super("Mammal");             // calls Animal(String)  
        System.out.println("Dog: " + breed);  
    }  
}  
  
new Dog();  
// Output:  
// Animal: Mammal  
// Dog: Labrador  
// Dog default
```

Q4. Can a constructor be private? When would you use it?

✓ PERFECT ANSWER

Yes, constructors can be private. Common use cases:

14. Singleton pattern — only one instance allowed.
15. Utility classes — prevent instantiation (e.g., Math class).
16. Static factory methods — controlled object creation.
17. Builder pattern — force usage of builder for construction.

```
class Singleton {  
    private static Singleton instance;  
  
    private Singleton() {                // Private — no external  
instantiation  
        System.out.println("Singleton created");  
    }  
  
    public static Singleton getInstance() {  
        if (instance == null) {  
            instance = new Singleton();  
        }  
        return instance;  
    }  
}  
  
Singleton s = Singleton.getInstance();  
// new Singleton(); // COMPILER ERROR — private constructor
```

Q5. Can a constructor be final, static, or abstract?

✓ PERFECT ANSWER

All three: NO. Reasoning:

- final → Constructors are not inherited, so 'final' to prevent overriding is meaningless.
- static → Constructors initialize instance state. Static means class-level, contradicting the purpose.
- abstract → An abstract method has no body and must be implemented. A constructor with no body would never initialize. Contradiction.

Q6. What is the default constructor? When is it provided?

✓ PERFECT ANSWER

The default constructor is a no-argument constructor automatically inserted by the compiler if and only if no other constructor is explicitly defined in the class. It calls `super()` and assigns default values to instance variables.

Critical: If you define any constructor (even parameterized), the compiler will NOT provide a default constructor.

```
class A {  
    // No constructor defined → compiler adds:  
    // A() { super(); }  
}  
  
class B {  
    B(int x) { }  
    // Compiler does NOT add default constructor.  
    // new B(); // COMPILE ERROR  
}
```

Q7. What is a copy constructor? Why does Java not provide one by default?

✓ PERFECT ANSWER

A copy constructor accepts another object of the same class and copies its field values into the new object. Unlike C++, Java does NOT provide a built-in copy constructor — the developer must write one explicitly.

Java provides alternatives: clone() method (via Cloneable interface) and serialization-based deep copy.

```
class Point {  
    int x, y;  
  
    Point(int x, int y) { this.x = x; this.y = y; }  
  
    // Copy constructor  
    Point(Point p) {  
        this.x = p.x;  
        this.y = p.y;  
    }  
}  
  
Point p1 = new Point(3, 4);  
Point p2 = new Point(p1);    // Independent copy
```

Q8. What is the order of execution: static block, instance block, constructor?

✓ PERFECT ANSWER

Execution order when first object is created:

18. Static variables initialized (default values, then explicit values).
19. Static block(s) execute — only once per class load, in declaration order.
20. Instance variables initialized.
21. Instance block(s) execute — every object creation, in declaration order.
22. Constructor body executes.

```
class Demo {  
    static { System.out.println("1. Static block"); }  
    { System.out.println("3. Instance block"); }  
  
    Demo() {  
        System.out.println("4. Constructor");  
    }  
  
    public static void main(String[] args) {  
        System.out.println("2. main starts");  
        new Demo();  
        new Demo();  
    }  
}  
  
// Output:  
// 1. Static block  
// 2. main starts  
// 3. Instance block  
// 4. Constructor  
// 3. Instance block  
// 4. Constructor
```

★ VVI: EXECUTION ORDER (Most-Asked)

Memorize: Static → main → Instance block → Constructor. Static runs once; instance runs per object.

Q9. Can we call a constructor explicitly?

✓ PERFECT ANSWER

Yes — but only from within another constructor of the same or parent class, using `this()` or `super()` respectively. You cannot call a constructor like a regular method from outside or from a non-constructor context.

```
class A {  
    A() { }  
    void method() {  
        // A();           // COMPILE ERROR — cannot call constructor directly  
        // this();        // COMPILE ERROR — only valid inside constructor  
    }  
  
    A(int x) {  
        this();           // Legal — first statement  
    }  
}
```

Q10. What is constructor overloading? Give a real-world example.

✓ PERFECT ANSWER

Constructor overloading is defining multiple constructors with different parameter lists in the same class. The compiler chooses the appropriate one based on the arguments passed.

Real use case: `StringBuilder` has overloaded constructors — `StringBuilder()`, `StringBuilder(int capacity)`, `StringBuilder(String str)`, `StringBuilder(CharSequence cs)`.

```
class Box {  
    double length, width, height;  
  
    Box() {  
        this(1, 1, 1);                // Default cube  
    }  
  
    Box(double side) {  
        this(side, side, side);        // Cube  
    }  
  
    Box(double l, double w, double h) {    // Cuboid  
        this.length = l;  
        this.width = w;  
        this.height = h;  
    }  
}  
  
Box b1 = new Box();                    // 1x1x1  
Box b2 = new Box(5);                  // 5x5x5  
Box b3 = new Box(2, 3, 4);            // 2x3x4
```

SUMMARY: Constructors

- Same name as class, no return type.
- Three types: default, parameterized, copy.
- Constructor chaining via `this()` or `super()` — must be the first statement.
- Cannot be final, static, or abstract.
- Compiler provides default constructor ONLY if no constructor is defined.

TOPIC 1.3: this KEYWORD

Q1. What is the 'this' keyword in Java? What are its uses?

✓ PERFECT ANSWER

'this' is an implicit reference variable that refers to the current object — the object on which the method or constructor is invoked. It is available in every non-static method and constructor.

Six primary uses:

23. Refer to current class instance variables (resolves shadowing by parameters).
24. Invoke current class method (implicit; rarely needed explicitly).
25. Invoke current class constructor — this().
26. Pass current object as argument to another method.
27. Return current object from a method (used in method chaining / builders).
28. Pass current object to another class constructor.

```
class Employee {
    int id;
    String name;

    Employee(int id, String name) {
        this.id = id;                // (1) resolve shadowing
        this.name = name;
    }

    Employee() {
        this(0, "Unknown");          // (3) call another constructor
    }

    Employee setName(String name) {
        this.name = name;
        return this;                 // (5) method chaining
    }

    void display() {
        Printer.print(this);         // (4) pass current object
    }
}
```

Q2. Can 'this' be used inside a static method? Why or why not?

✓ PERFECT ANSWER

No. 'this' refers to the current object instance, but static methods belong to the class — they are invoked without an instance. There is no 'current object' in a static context, so using 'this' would be meaningless and causes a compile-time error.

```
class Demo {  
    int x = 10;  
  
    static void method() {  
        // System.out.println(this.x); // COMPILER ERROR  
        // Cannot reference 'this' from a static context  
    }  
}
```

Q3. How does 'this' help in method chaining? Give an example.

✓ PERFECT ANSWER

Method chaining is the practice of invoking multiple methods on the same object in a single expression, by having each method return 'this'. This is the foundation of the Builder pattern and fluent APIs.

```
class StringBuilder2 {  
    private StringBuilder sb = new StringBuilder();  
  
    StringBuilder2 add(String s) {  
        sb.append(s);  
        return this;                // Enables chaining  
    }  
  
    StringBuilder2 newLine() {  
        sb.append("\n");  
        return this;  
    }  
  
    @Override  
    public String toString() {  
        return sb.toString();  
    }  
}  
  
String result = new StringBuilder2()  
    .add("Hello, ")  
    .add("Java")  
    .newLine()  
    .add("Welcome")  
    .toString();
```

Q4. What is the difference between 'this()' and 'super()'?

Feature	this()	super()
Refers to	Current class constructor	Parent class constructor

Feature	this()	super()
Purpose	Constructor chaining within class	Invoke parent constructor
Default	Not added by compiler	Implicitly added if not present
Position	First statement	First statement
Combined use	Cannot use both in same constructor	Cannot use both in same constructor

Q5. Is 'this' a keyword or a reference variable?**✓ PERFECT ANSWER**

'this' is a reserved keyword in Java that acts as a reference variable pointing to the current object. Technically, the JVM stores 'this' as a local variable in the method's stack frame — the first argument passed implicitly to every non-static method.

Q6. Can you reassign 'this' to a different object?**✓ PERFECT ANSWER**

No. 'this' is effectively final — it always refers to the current object on which the method was invoked. Attempting to assign a new value to 'this' causes a compile-time error.

```
class Demo {  
    void method() {  
        // this = new Demo();    // COMPILER ERROR — cannot assign to 'this'  
    }  
}
```

Q7. What happens if 'this' is used in a constructor before any field is initialized?**✓ PERFECT ANSWER**

'this' is available from the moment a constructor begins executing — even before fields receive their explicit values. At that point, instance variables hold their default values (0, null, false). Accessing fields via this is legal but may return defaults.

```
class Demo {  
    int x;  
    String s;  
  
    Demo() {  
        System.out.println(this.x);    // 0  
        System.out.println(this.s);    // null  
        this.x = 100;  
        System.out.println(this.x);    // 100  
    }  
}
```

Q8. How does 'this' enable the Builder pattern?

✓ PERFECT ANSWER

The Builder pattern uses 'this' returned from each setter to allow fluent, chainable configuration. It is the dominant pattern for constructing immutable objects with many optional parameters (e.g., `StringBuilder`, `Stream API`, Lombok `@Builder`).

```
class User {  
    private String name, email;  
    private int age;  
  
    static class Builder {  
        private User user = new User();  
  
        Builder name(String n)    { user.name = n;  return this; }  
        Builder email(String e)   { user.email = e; return this; }  
        Builder age(int a)        { user.age = a;   return this; }  
  
        User build() { return user; }  
    }  
}  
  
User u = new User.Builder()  
    .name("Md")  
    .email("md@example.com")  
    .age(21)  
    .build();
```

Q9. Can 'this' refer to an outer class instance from an inner class?

✓ PERFECT ANSWER

No — inside an inner class, 'this' refers to the inner class instance. To refer to the enclosing outer class instance, use `OuterClassName.this`.

```
class Outer {  
    int x = 10;  
  
    class Inner {  
        int x = 20;  
  
        void show() {  
            System.out.println(this.x);           // 20 (inner)  
            System.out.println(Outer.this.x);      // 10 (outer)  
        }  
    }  
}
```

Q10. What is the difference between local variable, instance variable, and 'this'?

✓ PERFECT ANSWER

'this' is a reference, not a variable category. It distinguishes between an instance variable and a local variable (or parameter) that share the same name.

```
class Demo {  
    int value;           // Instance variable  
  
    void set(int value) { // Parameter (local) shadows instance  
        // value = value; // Both refer to parameter – no effect on  
instance  
        this.value = value; // Resolves correctly: instance = parameter  
    }  
}
```



SUMMARY: this Keyword

- Refers to the current object; available only in non-static contexts.
- Six uses: field access, method call, constructor chain, pass as arg, return, builder.
- Cannot be reassigned; effectively final.
- Use OuterClass.this from inner classes to access enclosing instance.
- this() must be the first statement and cannot coexist with super().

TOPIC 1.4: static KEYWORD

Q1. What is the 'static' keyword? Where can it be applied?

✓ PERFECT ANSWER

'static' is a modifier that makes a member belong to the class itself rather than to any specific instance. Static members are shared across all instances and exist independently of any object.

Can be applied to:

29. Variables (class variables).
30. Methods (class methods).
31. Blocks (static initializer blocks).
32. Nested classes (only — top-level classes cannot be static).
33. Imports — `import static java.lang.Math.*;`

```
class Counter {  
    static int count = 0;           // Static variable – shared  
    int id;                         // Instance variable – unique per object  
  
    static {                       // Static block – runs once on class load  
        System.out.println("Class loaded");  
    }  
  
    Counter() {  
        count++;                   // Modifies shared variable  
        id = count;  
    }  
  
    static int getCount() {        // Static method  
        return count;  
    }  
}  
  
new Counter(); new Counter(); new Counter();  
System.out.println(Counter.getCount()); // 3
```

Q2. Difference between static variable and instance variable?

Feature	Static Variable	Instance Variable
Belongs to	Class	Object
Memory	Method area (one copy)	Heap (per object)
Access	ClassName.var or instance.var	instance.var only
Initialization	On class load	On object creation
Lifetime	Until class is unloaded	Until object is GC'd
Use case	Constants, counters, configs	Object-specific state

Q3. Can a static method access non-static members? Why or why not?

✓ PERFECT ANSWER

No — a static method cannot directly access non-static (instance) members because static methods do not have an implicit 'this' reference. There is no 'current object' to fetch instance state from.

Workaround: Pass an object reference as a parameter to access instance members.

```
class Demo {  
    int x = 10;           // instance  
    static int y = 20;    // static  
  
    static void method() {  
        System.out.println(y);    // OK  
        // System.out.println(x);  // COMPILE ERROR — needs an instance  
        Demo d = new Demo();  
        System.out.println(d.x);   // OK — explicit instance  
    }  
}
```

Q4. What is a static block? When does it execute?

✓ PERFECT ANSWER

A static block is a block of code enclosed in 'static { }' inside a class. It executes exactly once — when the class is loaded into JVM (before main() or any other static/instance code). Used for complex static variable initialization or one-time setup.

Order: Multiple static blocks execute in the order they appear in the source code.

```
class Config {  
    static Map<String, String> settings;  
  
    static {  
        settings = new HashMap<>();  
        settings.put("env", "prod");  
        settings.put("version", "1.0");  
        System.out.println("Config initialized");  
    }  
}
```

Q5. Can we override a static method?

✓ PERFECT ANSWER

No — static methods cannot be overridden. They are resolved at compile time based on the reference type (early binding). If a subclass defines a static method with the same signature, it is method Hiding, not overriding.

Test in interview: Polymorphism (runtime dispatch) does not apply to static methods.

```
class Parent {  
    static void show() { System.out.println("Parent static"); }  
}  
  
class Child extends Parent {  
    static void show() { System.out.println("Child static"); }    // HIDING  
}  
  
Parent p = new Child();  
p.show();           // Output: Parent static (compile-time, based on reference)
```

★ VVI: STATIC METHOD HIDING vs OVERRIDING

This is one of the most-asked tricky questions. Static = compile-time binding; instance methods = runtime binding. Method hiding $\neq$ overriding.

Q6. Why is the main method static?

✓ PERFECT ANSWER

The main method is static so that the JVM can invoke it WITHOUT creating an instance of the class. If main were non-static, the JVM would need an object — but creating that object would require executing another method first, creating a chicken-and-egg problem.

Q7. Can the main method be overloaded? Overridden? Final?

✓ PERFECT ANSWER

- Overloaded: YES — you can have multiple main methods; JVM calls only the one with String[] args signature.
- Overridden: NO — main is static, and static methods cannot be overridden.
- Final: YES — making main final is legal (no impact since static).

```
public class Demo {  
    public static void main(String[] args) {                // JVM entry  
        System.out.println("Standard main");  
        main(10);                                           // Manual call  
    }  
  
    public static void main(int x) {                        // Overloaded  
        System.out.println("Overloaded with int: " + x);  
    }  
}
```

Q8. What is the difference between static method and instance method?

Feature	Static Method	Instance Method
Belongs to	Class	Object
this reference	Not available	Available
Access to instance members	Only via passed reference	Direct
Binding	Compile-time (early)	Runtime (late)
Override	Not possible (hiding only)	Possible
Memory	Method area	Method area + invoked on heap object

Q9. What is a static nested class? How is it different from an inner class?

✓ PERFECT ANSWER

A static nested class is a class declared with 'static' inside another class. It does NOT have access to instance members of the enclosing class — only static ones. It behaves like a regular top-level class but is logically grouped with its enclosing class.

Inner class (non-static nested): Has an implicit reference to the enclosing instance; can access all members including private.

```
class Outer {  
    static int s = 10;  
    int i = 20;  
  
    static class StaticNested {  
        void show() {  
            System.out.println(s);    // OK  
            // System.out.println(i); // ERROR — no enclosing instance  
        }  
    }  
  
    class Inner {  
        void show() {  
            System.out.println(s);    // OK  
            System.out.println(i);    // OK  
        }  
    }  
}  
  
Outer.StaticNested sn = new Outer.StaticNested();    // No outer needed  
Outer.Inner in = new Outer().new Inner();            // Outer needed
```

Q10. Why are constants typically declared 'public static final'?**✓ PERFECT ANSWER**

- public — accessible everywhere.
- static — one shared copy per class, no need to create an instance.
- final — value cannot be reassigned.

This combination produces a true compile-time constant with the minimum memory footprint and maximum performance. Naming convention: ALL_UPPER_CASE.

```
class MathConstants {  
    public static final double PI = 3.14159265359;  
    public static final int MAX_RETRIES = 5;  
    public static final String APP_NAME = "MyApp";  
}  
  
double area = MathConstants.PI * r * r;
```



SUMMARY: static Keyword

- Belongs to class, not object — one copy shared across all instances.
- Can be applied to variables, methods, blocks, nested classes, imports.
- Static methods cannot access instance members directly or use this/super.
- Static methods are HIDDEN (not overridden) in subclasses — compile-time binding.
- Static block runs once on class load; main is static for JVM entry point.

PART 2: INTERMEDIATE LEVEL

This section covers the four pillars of OOP — Inheritance, Polymorphism, Encapsulation, Abstraction — plus access modifiers. These topics dominate intermediate Java interviews and are asked in 90%+ of backend developer interviews.

TOPIC 2.1: INHERITANCE

Q1. What is inheritance? Why is it used? Types of inheritance in Java.

✓ PERFECT ANSWER

Inheritance is a mechanism where one class (child/subclass/derived) acquires the properties and behaviors of another class (parent/superclass/base). It is implemented in Java using the 'extends' keyword. It promotes code reusability and establishes an IS-A relationship.

Five types of inheritance:

- 34. Single — one parent, one child ($A \rightarrow B$).
- 35. Multilevel — chain of inheritance ($A \rightarrow B \rightarrow C$).
- 36. Hierarchical — one parent, multiple children ($A \rightarrow B, A \rightarrow C$).
- 37. Multiple — multiple parents (NOT supported via classes in Java; supported via interfaces).
- 38. Hybrid — combination of the above (only via interfaces in Java).

```
// Single inheritance

class Animal {
    void eat() { System.out.println("Eating..."); }
}

class Dog extends Animal {
    void bark() { System.out.println("Barking..."); }
}

// Multilevel inheritance

class Puppy extends Dog {
    void weep() { System.out.println("Weeping..."); }
}

// Hierarchical inheritance

class Cat extends Animal {
    void meow() { System.out.println("Meowing..."); }
}

Puppy p = new Puppy();
p.eat();    // From Animal
p.bark();   // From Dog
p.weep();   // From Puppy
```



REAL-WORLD ANALOGY

"Inheritance is like a child inheriting traits from parents. A child has its parents' eye color (inherited) plus its own unique features (new methods). Multilevel is grandparent → parent → child."



WHAT INTERVIEWER TESTS

- IS-A relationship understanding.
- Knowledge of all five types and which Java supports natively.
- Reasoning for why Java does NOT support multiple inheritance via classes.
- Ability to design clean class hierarchies.



FOLLOW-UP QUESTIONS

- Why doesn't Java support multiple inheritance through classes? → Diamond problem (ambiguity).
- How does Java solve the diamond problem? → Through interfaces with default methods (Java 8+) using rules.
- What is the super keyword? → Reference to parent class members.
- Can a final class be inherited? → No.
- Can private methods be inherited? → No — they exist in the parent but not visible/accessable.



COMMON MISTAKES

- Claiming Java supports multiple inheritance through classes.
- Confusing inheritance (IS-A) with composition (HAS-A).
- Forgetting that constructors are NOT inherited.
- Always mention the diamond problem when discussing multiple inheritance.

Q2. What is the diamond problem? How does Java solve it?



PERFECT ANSWER

The diamond problem arises when a class inherits from two classes that both inherit from a common ancestor and override the same method. The compiler cannot determine which version to use.

Java's solution:

- Multiple inheritance through CLASSES is forbidden — eliminates the ambiguity entirely.
- Multiple inheritance through INTERFACES is allowed — but if a default method conflict occurs, the implementing class MUST override the method to resolve the ambiguity explicitly.

```
interface A {  
    default void show() { System.out.println("A"); }  
}  
  
interface B {  
    default void show() { System.out.println("B"); }  
}  
  
class C implements A, B {  
    // Must override to resolve conflict  
    @Override  
    public void show() {  
        A.super.show();    // Explicitly choose A's version  
    }  
}
```

Q3. What is the difference between extends and implements?

Feature	extends	implements
Used with	class extends class / interface extends interface	class implements interface
Multiple allowed	No (only one class can be extended)	Yes (multiple interfaces)
Inherits	Properties + methods + constructors	Method signatures (contract)
Override needed	Optional (parent methods usable as-is)	Mandatory for all abstract methods

Q4. What is the 'super' keyword? What are its uses?

✓ PERFECT ANSWER

Three uses:

- 39. Access parent class fields hidden by child class — `super.field`.
- 40. Invoke parent class methods overridden by child — `super.method()`.
- 41. Invoke parent class constructor — `super(args)`. Must be first statement.

```
class Vehicle {
    String type = "Vehicle";

    Vehicle(String t) { this.type = t; }

    void start() { System.out.println("Vehicle started"); }
}

class Car extends Vehicle {
    String type = "Car";

    Car() {
        super("Sedan"); // (3) parent constructor
    }

    @Override
    void start() {
        super.start(); // (2) parent method
        System.out.println("Car-specific start");
        System.out.println(super.type); // (1) parent field
    }
}
```

Q5. What is method overriding? Rules of method overriding.

✓ PERFECT ANSWER

Method overriding occurs when a subclass provides a specific implementation of a method already defined in its parent class, with the same signature. It enables runtime polymorphism.

Strict rules:

42. Method name and parameter list must be IDENTICAL.
43. Return type must be same or covariant (subtype of parent's return type).
44. Access modifier cannot be MORE restrictive (can be same or wider).
45. Cannot override: static, final, private, or constructor.
46. Checked exceptions thrown can only be the same or subclass of parent's declared exceptions.
47. `@Override` annotation recommended (catches typos at compile time).

```
class Animal {  
    protected Animal getInstance() throws IOException {  
        return new Animal();  
    }  
}  
  
class Dog extends Animal {  
    // Covariant return: Dog is subtype of Animal – LEGAL  
    @Override  
    public Dog getInstance() throws FileNotFoundException {  
        // Wider access (public > protected) – LEGAL  
        // Narrower exception (FileNotFoundException extends IOException) –  
LEGAL  
        return new Dog();  
    }  
}
```

Q6. Can we override private and final methods?

✓ PERFECT ANSWER

private methods: NO. Private methods are not visible to subclasses. Defining a same-signature method in a subclass creates an entirely new method (not an override) — no `@Override` allowed.

final methods: NO. The 'final' modifier explicitly prevents overriding. Attempting it causes a compile error.

```
class A {
    private void show() { System.out.println("A private"); }
    final void display() { System.out.println("A final"); }
}

class B extends A {
    // private void show() {}    // NOT an override — new method
    // void display() {}        // COMPILE ERROR — cannot override final
}
```

Q7. What is the difference between method overloading and method overriding?

Feature	Overloading	Overriding
When resolved	Compile-time	Runtime
Where	Same class (or inherited)	Parent–Child
Method signature	Must differ in parameters	Must be identical
Return type	Can differ	Same or covariant
Access modifier	Any	Cannot be more restrictive
Polymorphism type	Static / Compile-time	Dynamic / Runtime
<code>@Override</code>	Not applicable	Recommended
Inheritance required	No	Yes

Q8. What is covariant return type?

✓ PERFECT ANSWER

Since Java 5, an overriding method can return a SUBTYPE of the return type declared in the parent method. This is called covariant return type and reduces the need for casting.

```
class Animal { }

class Dog extends Animal { }

class AnimalShelter {
    Animal adopt() { return new Animal(); }
}

class DogShelter extends AnimalShelter {
    @Override
    Dog adopt() { return new Dog(); }    // Dog is subtype of Animal — LEGAL
}

Dog d = new DogShelter().adopt();      // No cast needed
```

Q9. What is the order of constructor execution in inheritance?

✓ PERFECT ANSWER

In an inheritance chain, parent constructor ALWAYS executes before child constructor. This is enforced by an implicit (or explicit) `super()` call as the first statement in every constructor.

Order for an object of class C (where $C \rightarrow B \rightarrow A \rightarrow \text{Object}$):

- 48. `Object()`
- 49. `A()` — static blocks, instance blocks, body
- 50. `B()` — static blocks, instance blocks, body
- 51. `C()` — static blocks, instance blocks, body

Static blocks execute only on FIRST class load (top-down: $A \rightarrow B \rightarrow C$).

```
class A {  
    A() { System.out.println("A constructor"); }  
}  
  
class B extends A {  
    B() {  
        // super(); – implicitly added  
        System.out.println("B constructor");  
    }  
}  
  
class C extends B {  
    C() {  
        System.out.println("C constructor");  
    }  
}  
  
new C();  
// Output:  
// A constructor  
// B constructor  
// C constructor
```

Q10. What is the difference between IS-A and HAS-A relationship?

Aspect	IS-A (Inheritance)	HAS-A (Composition)
Keyword	extends / implements	Object reference as field
Coupling	Tight	Loose
Reusability	Through inheritance hierarchy	Through delegation
Example	Dog IS-A Animal	Car HAS-A Engine

Aspect	IS-A (Inheritance)	HAS-A (Composition)
Flexibility	Fixed at compile-time	Can change at runtime
Encapsulation	Exposes parent internals	Hides component details
Recommendation	Use when true subtype relationship	Prefer for code reuse — GoF principle

Q11. Can we inherit constructors in Java?**✓ PERFECT ANSWER**

NO. Constructors are NOT inherited. Each class defines its own constructors. However, a subclass constructor must invoke a parent constructor (explicitly via `super()` or implicitly via the compiler) — this is constructor chaining, not inheritance.

Q12. What is multilevel inheritance? Give an example.**✓ PERFECT ANSWER**

Multilevel inheritance is a chain where a class inherits from a class that itself inherits from another class. Each level adds new behavior on top of inherited behavior.

```
class Vehicle {  
    void start() { System.out.println("Vehicle starts"); }  
}  
  
class Car extends Vehicle {  
    void wheels() { System.out.println("4 wheels"); }  
}  
  
class SportsCar extends Car {  
    void topSpeed() { System.out.println("300 km/h"); }  
}  
  
SportsCar sc = new SportsCar();  
sc.start();           // From Vehicle  
sc.wheels();          // From Car  
sc.topSpeed();        // Own
```

Q13. What is hierarchical inheritance? When is it useful?

✓ PERFECT ANSWER

In hierarchical inheritance, multiple subclasses inherit from a single parent class. It is useful when many entities share common attributes/behavior but specialize differently.

Example use case: A Shape parent with Circle, Rectangle, Triangle subclasses — all share color/position but compute area differently.

Q14. What is a sealed class? (Java 17+)

✓ PERFECT ANSWER

A sealed class (Java 17) restricts WHICH classes can extend it using the 'permits' clause. This enables controlled inheritance and exhaustive pattern matching.

Subclasses of a sealed class must be: final, sealed, or non-sealed.

```
public sealed class Shape permits Circle, Square, Triangle { }

public final class Circle extends Shape { }

public final class Square extends Shape { }

public non-sealed class Triangle extends Shape { }    // Open for further
extension
```

Q15. When should you NOT use inheritance?

✓ PERFECT ANSWER

- When there's no clear IS-A relationship — favor composition (HAS-A).
- When you only need to reuse code, not establish a type hierarchy.
- When the parent's API exposes implementation details you'd rather hide.
- When you'd need to override most methods to make them work differently — sign of wrong abstraction.
- Classic guidance: "Favor composition over inheritance" — Effective Java, Joshua Bloch.



SUMMARY: Inheritance

- Java supports single, multilevel, and hierarchical via classes; multiple/hybrid via interfaces.
- Diamond problem prevents class-based multiple inheritance.
- Constructors are not inherited; parent constructor runs first.
- Cannot override private, final, or static methods.
- Favor composition over inheritance — Effective Java principle.

TOPIC 2.2: POLYMORPHISM

Q1. What is polymorphism? Explain its two types with examples.

✓ PERFECT ANSWER

Polymorphism (from Greek: many forms) means an entity can take multiple forms — same interface, different behaviors. In Java, it manifests in two ways:

1. COMPILE-TIME POLYMORPHISM (Static / Method Overloading):

- Same method name with different parameter lists in the same class.
- Resolved by the compiler based on method signature.
- Also called early binding or static dispatch.

2. RUNTIME POLYMORPHISM (Dynamic / Method Overriding):

- Child class redefines a parent class method with same signature.
- JVM resolves at runtime based on actual object type, not reference type.
- Also called late binding or dynamic dispatch.

```
// COMPILE-TIME (Overloading)

class Calculator {

    int add(int a, int b)          { return a + b; }

    int add(int a, int b, int c)   { return a + b + c; }

    double add(double a, double b) { return a + b; }

}

Calculator c = new Calculator();

c.add(2, 3);           // Compiler picks int version
c.add(2.5, 3.5);       // Compiler picks double version


// RUNTIME (Overriding)

class Animal {

    void sound() { System.out.println("Generic sound"); }

}

class Dog extends Animal {

    @Override

    void sound() { System.out.println("Bark"); }

}

class Cat extends Animal {

    @Override

    void sound() { System.out.println("Meow"); }

}

Animal a;

a = new Dog();    a.sound();    // "Bark" - runtime decision
a = new Cat();    a.sound();    // "Meow" - runtime decision
```



REAL-WORLD ANALOGY

"A person behaves differently in different roles: student at college, employee at office, customer at a shop. Same person — many forms. Same goes for a method: same name, different behavior depending on context."



WHAT INTERVIEWER TESTS

- Clear distinction between the two types.
- Understanding of static vs dynamic binding.
- Ability to write working code for both.
- Knowing that variables/fields are not polymorphic (only methods).



FOLLOW-UP QUESTIONS

- Are static methods polymorphic? → NO — static methods use hiding, not overriding.
- Are fields polymorphic? → NO — field access uses reference type, not object type.
- What is upcasting/downcasting? → Casting reference up/down the inheritance tree.
- Can constructors be overloaded? → YES. Can they be overridden? → NO.
- Is method overloading runtime or compile-time? → Compile-time.



COMMON MISTAKES

-  Saying 'overloading and overriding are the same with different names'.
-  Confusing static method hiding with polymorphism.
-  Thinking field access is polymorphic.
-  Always pair the explanation with a code example showing both upcasting and runtime dispatch.

Q2. What is method overloading? What are its rules?



PERFECT ANSWER

Method overloading is defining multiple methods with the same name but different parameter lists within the same class (or inherited).

Methods can differ in:

52. Number of parameters.
53. Types of parameters.
54. Order of parameters.

Return type alone is NOT sufficient — methods cannot be overloaded by return type only.

```
class Print {  
    void show(int x)          { System.out.println("int"); }  
    void show(double x)       { System.out.println("double"); }  
    void show(int x, int y)    { System.out.println("two ints"); }  
    void show(String s, int x) { System.out.println("String, int"); }  
    void show(int x, String s) { System.out.println("int, String"); }  
  
    // int show(int x) { return x; } // ERROR — same signature  
}
```

Q3. Can main() be overloaded?

✓ PERFECT ANSWER

YES — main can be overloaded like any static method. However, the JVM will only call the standard signature: `public static void main(String[] args)`. Other overloaded versions must be called manually.

```
public class Demo {  
  
    public static void main(String[] args) {  
  
        System.out.println("Default main");  
  
        main(10);  
  
        main("Hello");  
  
    }  
  
    public static void main(int x) {  
  
        System.out.println("int main: " + x);  
  
    }  
  
    public static void main(String s) {  
  
        System.out.println("String main: " + s);  
  
    }  
  
}
```

Q4. What is dynamic method dispatch?

✓ PERFECT ANSWER

Dynamic method dispatch is the mechanism by which a call to an overridden method is resolved at RUNTIME based on the actual object type, not the reference type. This is the core mechanism enabling runtime polymorphism in Java.

Mechanism: JVM maintains a virtual method table (vtable) for each class. When a method is called on a reference, JVM looks up the actual object's class vtable to find the right implementation.

```
Animal a = new Dog();  
  
a.sound();  
  
// 1. Reference 'a' is of type Animal (compile-time type).  
// 2. Actual object is Dog.  
// 3. JVM looks up Dog's vtable → finds Dog.sound().  
// 4. Output: "Bark"
```

Q5. What is upcasting and downcasting?

✓ PERFECT ANSWER

Upcasting: Casting a subclass reference to a superclass type. Implicit and always safe.

Downcasting: Casting a superclass reference back to a subclass. Must be explicit. Throws `ClassCastException` at runtime if the object is not actually of that subclass.

```
Animal a = new Dog();           // Upcasting (implicit)

// a.bark();                     // ERROR - Animal reference doesn't know bark()

if (a instanceof Dog) {
    Dog d = (Dog) a;             // Downcasting (explicit)
    d.bark();                    // OK
}

Animal a2 = new Cat();

Dog d2 = (Dog) a2;              // Compiles OK, but ClassCastException at
runtime
```

Q6. What is the instanceof operator? When is it used?

✓ PERFECT ANSWER

'instanceof' tests whether an object is an instance of a particular class, subclass, or interface. It returns true/false. Used to safely downcast and avoid `ClassCastException`.

Java 16+ pattern matching: instanceof can declare a binding variable in one step.

```
Object obj = "Hello";

// Traditional
if (obj instanceof String) {
    String s = (String) obj;
    System.out.println(s.length());
}

// Pattern matching (Java 16+)
if (obj instanceof String s) {
    System.out.println(s.length());    // 's' auto-cast
}
```

Q7. Why are fields not polymorphic in Java?

✓ PERFECT ANSWER

Java fields are resolved at compile time based on the REFERENCE TYPE, not the actual object type. This is by design — field access has zero runtime overhead compared to virtual method dispatch.

Practical implication: Always use getters/setters for fields you may need to override behavior for.

```

class A {
    String name = "A";

    String getName() { return name; }
}

class B extends A {
    String name = "B";

    @Override
    String getName() { return name; }
}

A obj = new B();

System.out.println(obj.name);           // "A"    - field access uses reference
type

System.out.println(obj.getName());      // "B"    - method dispatch uses object
type

```

★ VVI: FIELDS vs METHODS in Polymorphism

Fields → compile-time (reference type wins). Methods → runtime (object type wins). This is a frequent gotcha question.

Q8. What is the difference between static and dynamic binding?

Feature	Static Binding (Early)	Dynamic Binding (Late)
When resolved	Compile-time	Runtime
Applies to	static, final, private, overloaded	Overridden instance methods
Mechanism	Direct call by signature	Virtual method table lookup
Performance	Faster	Slight overhead
Polymorphism type	Compile-time	Runtime
Example	Method overloading	Method overriding

Q9. Can we overload methods that differ only in static keyword?

✓ PERFECT ANSWER

NO. Methods cannot be overloaded based solely on the static modifier. The parameter list MUST differ. The compiler treats both as having the same signature.

```
class Demo {  
    void method(int x) { }  
    // static void method(int x) { }    // COMPILE ERROR — duplicate method  
}
```

Q10. What happens if a parent class has a private method and the child defines the same method?

✓ PERFECT ANSWER

This is NOT overriding — it's a completely new method in the child class. The parent's private method is invisible to the child. The two methods are unrelated.

Compiler will not complain: `@Override` annotation, however, would cause a compile error if used.

Q11. What are real-world examples of polymorphism in Java standard library?

✓ PERFECT ANSWER

- 55. Collections framework: `List<E>` reference can hold `ArrayList`, `LinkedList`, `Vector` — same API, different implementations.
- 56. Stream API: `stream.filter().map().collect()` — works on any `Collection`.
- 57. `Comparable` / `Comparator` — `compareTo()` overridden per type.
- 58. Object methods: `toString()`, `equals()`, `hashCode()` — every class overrides these.
- 59. Servlet API: `doGet()`, `doPost()` overridden per servlet.

Q12. Why is overriding `equals()` also requiring overriding `hashCode()`?

✓ PERFECT ANSWER

The Java contract states: if two objects are equal (via equals), they **MUST** have the same hashCode. Hash-based collections (HashMap, HashSet) rely on this to find buckets and detect equality.

Violating this contract causes: Objects not found in HashSet/HashMap, duplicate keys in maps, broken set operations.

```
class Employee {  
    int id;  
    String name;  
  
    @Override  
    public boolean equals(Object o) {  
        if (this == o) return true;  
        if (!(o instanceof Employee)) return false;  
        Employee e = (Employee) o;  
        return id == e.id && Objects.equals(name, e.name);  
    }  
  
    @Override  
    public int hashCode() {  
        return Objects.hash(id, name);  
    }  
}
```

Q13. What is the use of the @Override annotation?

✓ PERFECT ANSWER

@Override is a compile-time hint telling the compiler that the annotated method is intended to override a parent method. If the method does not actually override (e.g., due to a typo or wrong signature), the compiler reports an error.

Benefits:

- Catches subtle bugs (typos, wrong parameter types).
- Improves readability — clear signal of intent.
- Maintains correctness as parent classes evolve.

Q14. Can constructors be polymorphic?

✓ PERFECT ANSWER

NO — constructors are not inherited, so they cannot be overridden. However, constructors CAN be overloaded (multiple constructors with different signatures), which is a form of compile-time polymorphism.

Q15. What is the difference between virtual functions in C++ and methods in Java?

✓ PERFECT ANSWER

In C++, methods are NOT virtual by default; you must explicitly mark them 'virtual' for dynamic dispatch. In Java, ALL non-static, non-private, non-final methods are virtual by default. Java has no 'virtual' keyword — it is implicit.



SUMMARY: Polymorphism

- Two types: compile-time (overloading) and runtime (overriding).
- Methods are polymorphic; fields and static methods are NOT.
- Dynamic dispatch resolves overridden methods at runtime via vtable.
- Upcasting is implicit; downcasting requires explicit cast and is risky without instanceof.
- Always pair equals() override with hashCode() override.

TOPIC 2.3: ENCAPSULATION

Q1. What is encapsulation? How is it achieved in Java?

✓ PERFECT ANSWER

Encapsulation is the bundling of data (fields) and the methods that operate on that data into a single unit (class), while restricting direct access to internal state. It enforces data hiding and controlled access.

How to achieve in Java:

60. Declare instance variables as private.
61. Provide public getter and setter methods for controlled access.
62. Add validation in setters to enforce invariants.
63. Make the class final if no extension is intended (advanced encapsulation).

```
public class BankAccount {  
    private double balance;           // Hidden  
    private String accountNumber;  
  
    public BankAccount(String accNum) {  
        this.accountNumber = accNum;  
    }  
  
    public double getBalance() {      // Read access  
        return balance;  
    }  
  
    public void deposit(double amount) {  
        if (amount <= 0) {  
            throw new IllegalArgumentException("Amount must be positive");  
        }  
        balance += amount;  
    }  
  
    public boolean withdraw(double amount) {  
        if (amount <= 0 || amount > balance) return false;  
        balance -= amount;  
        return true;  
    }  
  
    // No setter for accountNumber – immutable after construction  
}
```

**REAL-WORLD ANALOGY**

"A medicine capsule wraps the active ingredient inside a protective coating — you can use it via the prescribed dose (the public API) but cannot directly access the chemical compound (the private fields)."

**WHAT INTERVIEWER TESTS**

- Distinction between encapsulation and abstraction (often confused).
- Understanding of access modifiers and their role.
- Ability to design classes with proper invariants.
- Awareness of read-only / write-only fields.

**FOLLOW-UP QUESTIONS**

- How is encapsulation different from abstraction? → Encapsulation = hiding data; Abstraction = hiding implementation complexity.
- Can a class without private fields be encapsulated? → Technically not fully — public fields violate data hiding.
- What is a read-only / write-only field? → Only getter (read-only) or only setter (write-only).
- Why are JavaBeans the textbook example of encapsulation? → They strictly follow private-field + getter/setter convention.

**COMMON MISTAKES**

-  Equating encapsulation with just using private fields — encapsulation is the entire principle of controlled access.
-  Exposing mutable internal collections via getters (e.g., return list directly — caller can modify it).
-  Return defensive copies for mutable fields.

Q2. What is the difference between encapsulation and abstraction?

Feature	Encapsulation	Abstraction
Concept	Data hiding	Implementation hiding
Focus	How — data + methods bundled	What — essential behavior exposed
Achieved by	private fields + public getters/setters	abstract classes + interfaces
Level	Implementation level	Design level
Example	BankAccount with private balance	Vehicle interface with start() method
Question answered	How is data protected?	What does the object do?

Q3. How do you create a read-only and write-only class?

✓ PERFECT ANSWER

Read-only: Provide only getters, no setters.

Write-only: Provide only setters, no getters.

```
// Read-only class
final class Constants {
    private final String version;

    Constants(String v) { this.version = v; }

    public String getVersion() { return version; }
    // No setter
}

// Write-only class (rare; used for logging/auditing)
class AuditLog {
    private List<String> entries = new ArrayList<>();

    public void log(String entry) { entries.add(entry); }
    // No getter – entries cannot be read externally
}
```

Q4. What is the role of getters and setters? Are they always necessary?

✓ PERFECT ANSWER

Getters and setters provide controlled access to private fields. They allow:

- Validation in setters (reject invalid values).
- Computed properties in getters (derive on access).
- Lazy initialization.
- Internal representation changes without breaking callers.
- Logging/auditing of access.

Are they always necessary? NO. For immutable objects, only getters needed. For pure data carriers, consider Java Records (Java 14+).

Q5. What is an immutable class? How do you create one?

✓ PERFECT ANSWER

An immutable class is one whose instances cannot be modified after construction. All state is set at creation time and never changes.

Rules to make a class immutable:

64. Declare class as final (prevent subclassing that adds mutability).
65. All fields private and final.
66. No setter methods.
67. Initialize all fields via constructor.
68. If fields hold mutable objects, return defensive copies in getters and accept defensive copies in constructor.

```
public final class ImmutablePoint {  
    private final int x;  
    private final int y;  
    private final List<String> tags;  
  
    public ImmutablePoint(int x, int y, List<String> tags) {  
        this.x = x;  
        this.y = y;  
        this.tags = new ArrayList<>(tags);    // Defensive copy IN  
    }  
  
    public int getX() { return x; }  
    public int getY() { return y; }  
  
    public List<String> getTags() {  
        return new ArrayList<>(tags);    // Defensive copy OUT  
    }  
}
```

★ VVI: IMMUTABLE CLASS — THE 5 RULES

final class + private final fields + no setters + constructor init + defensive copies. Memorize this — asked in nearly every interview.

Q6. Why is String immutable in Java?

✓ PERFECT ANSWER

69. String pool optimization — multiple references share one object safely.
70. Thread safety — no synchronization needed.
71. Security — used for class names, URLs, file paths, passwords; mutation could be exploited.
72. hashCode caching — String's hashCode is computed once and reused; only possible if state cannot change.
73. ClassLoader integrity — class names are Strings; mutation would break the JVM.

Q7. What happens if a class has both encapsulation and inheritance?

✓ PERFECT ANSWER

Encapsulation works seamlessly with inheritance. Subclasses inherit the protected/public API but cannot directly access private fields — they must use inherited getters/setters. This preserves the parent's invariants.

```
class Account {
    private double balance;                // Hidden from subclasses

    protected double getBalance() { return balance; }
    protected void setBalance(double b) {
        if (b < 0) throw new IllegalArgumentException();
        balance = b;
    }
}

class SavingsAccount extends Account {
    void addInterest(double rate) {
        // balance += balance * rate;        // ERROR — private
        setBalance(getBalance() * (1 + rate)); // Uses inherited API
    }
}
```

Q8. What is a defensive copy? Why is it important?

✓ PERFECT ANSWER

A defensive copy is a fresh copy of a mutable object passed in or out of a class, preventing external code from modifying the internal state directly.

Without defensive copies, encapsulation can be broken:

```
// BROKEN encapsulation

class Schedule {
    private Date startDate;

    public Schedule(Date d) { this.startDate = d; }

    public Date getStartDate() { return startDate; }
}

Date d = new Date();
Schedule s = new Schedule(d);
d.setTime(0); // External mutation BREAKS Schedule's
state

// CORRECT – defensive copies

class Schedule {
    private final Date startDate;

    public Schedule(Date d) {
        this.startDate = new Date(d.getTime()); // Copy IN
    }

    public Date getStartDate() {
        return new Date(startDate.getTime()); // Copy OUT
    }
}
```

Q9. What are JavaBeans? How do they relate to encapsulation?

✓ PERFECT ANSWER

JavaBeans are reusable software components that follow a strict naming convention:

- Public no-arg constructor.
- Private fields with public getXxx()/setXxx() methods.
- Implement Serializable.
- Follow naming conventions for properties.

They are the textbook example of encapsulation and integrate seamlessly with frameworks like Spring (bean injection), JSF, JPA.

Q10. What is a Java Record? How does it relate to encapsulation?

✓ PERFECT ANSWER

Java Records (Java 14 preview, 16 standard) are compact, immutable data carriers. The compiler auto-generates: private final fields, public accessor methods, constructor, equals(), hashCode(), toString().

Records ENFORCE encapsulation: All fields are private final; no setters; class is implicitly final.

```
// One line — fully encapsulated, immutable
public record Point(int x, int y) { }

// Equivalent to ~30 lines of traditional encapsulation
Point p = new Point(3, 4);

p.x(); // Auto-generated accessor
// p.x = 5; // ERROR — final field
```

Q11. What are the benefits of encapsulation?

✓ PERFECT ANSWER

74. Data security — validation prevents invalid states.
75. Flexibility — internal implementation can change without affecting users.
76. Maintainability — clear API boundaries.
77. Testability — controlled access points easy to mock.
78. Read-only / write-only fields possible.
79. Easier debugging — all state mutations go through one path.

Q12. How does encapsulation help in unit testing?

✓ PERFECT ANSWER

Encapsulated classes expose a stable public API and hide internal state. Unit tests target the API contract, not the implementation. When internals change, tests still pass — making refactoring safe. Mocking frameworks (Mockito) rely on this clean API surface to substitute dependencies.



SUMMARY: Encapsulation

- Bundle data + methods, hide internal state via private fields.
- Provide controlled access through getters/setters with validation.
- Use defensive copies for mutable fields to prevent leaks.
- Immutable classes are the pinnacle of encapsulation.
- Java Records (14+) automate encapsulation for data carriers.

TOPIC 2.4: ABSTRACTION

Q1. What is abstraction? How is it achieved in Java?

✓ PERFECT ANSWER

Abstraction is the principle of hiding implementation complexity and exposing only essential features to the user. The user knows WHAT the object does, not HOW it does it.

Achieved in Java through:

80. Abstract classes (0% to 100% abstraction).
81. Interfaces (100% abstraction before Java 8; partial since).

```
abstract class Vehicle {  
    abstract void start();                // What – no how  
  
    void displayInfo() {                  // Concrete method  
        System.out.println("Vehicle info");  
    }  
}  
  
class Car extends Vehicle {  
    @Override  
    void start() {  
        // HOW: turn key, ignite engine, start fuel injection...  
        System.out.println("Car started");  
    }  
}  
  
Vehicle v = new Car();  
v.start();                               // Caller doesn't know the HOW
```



REAL-WORLD ANALOGY

"Driving a car: you turn the key (essential feature); you don't need to know how the spark plug, fuel injector, and ECU coordinate (implementation hidden)."



WHAT INTERVIEWER TESTS

- Difference between abstraction and encapsulation.
- When to choose interface vs abstract class.
- Knowledge of abstract methods, abstract classes, and their constraints.
- Real-world examples in the JDK.

+ FOLLOW-UP QUESTIONS

- Can an abstract class have a constructor? → Yes — called via `super()` from concrete subclass.
- Can an abstract method be private? → No — must be visible to be overridden.
- Can an abstract class have no abstract methods? → Yes — but unusual; typically used to prevent instantiation.
- Difference between abstract method and concrete method? → Abstract has no body; concrete does.

Q2. What is an abstract class? When should you use one?

✓ PERFECT ANSWER

An abstract class is a class declared with the 'abstract' keyword. It may contain abstract methods (without body) and concrete methods (with body). It cannot be instantiated directly.

Use an abstract class when:

- Related classes share substantial common code.
- You need to provide a default implementation that subclasses can extend.
- You need to declare non-public/protected members.
- You need to maintain state (instance fields) across subclasses.
- Your hierarchy represents an IS-A relationship.

```
abstract class Shape {  
    String color;                // State  
  
    Shape(String color) { this.color = color; }  
  
    abstract double area();      // Must override  
    abstract double perimeter(); // Must override  
  
    void describe() {           // Shared behavior  
        System.out.println(color + " shape with area " + area());  
    }  
}  
  
class Circle extends Shape {  
    double radius;  
  
    Circle(String color, double r) {  
        super(color);  
        this.radius = r;  
    }  
  
    @Override double area()      { return Math.PI * radius * radius; }  
    @Override double perimeter() { return 2 * Math.PI * radius; }  
}
```

Q3. Can an abstract class have constructors? Why?

✓ PERFECT ANSWER

YES — an abstract class can (and usually does) have constructors. Although the abstract class cannot be instantiated directly, its constructors are invoked via `super()` when a concrete subclass is constructed. They initialize the inherited state.

Q4. Can an abstract class have no abstract methods?

✓ PERFECT ANSWER

YES. An abstract class with zero abstract methods is legal. Reasons to use it:

- Prevent direct instantiation (e.g., utility base classes).
- Provide a partial template that future subclasses may extend.
- Force subclassing for architectural reasons.

Q5. Can abstract methods be private, static, or final?

✓ PERFECT ANSWER

- **private:** NO — abstract methods must be overridden by subclasses; private methods are invisible to subclasses.
- **static:** NO — static methods belong to the class and cannot be overridden.
- **final:** NO — final prevents overriding; abstract requires overriding. Contradiction.

Abstract methods must be at least package-private and inheritable.

Q6. What is the difference between abstract method and concrete method?

Feature	Abstract Method	Concrete Method
Body	No body (only signature)	Has implementation
Keyword	abstract	(none required)
Container	abstract class or interface	Any class
Override	Mandatory in subclass	Optional
Purpose	Define contract	Provide implementation

Q7. Can we declare a method abstract and final together?

✓ PERFECT ANSWER

NO. These modifiers are mutually exclusive:

- `abstract` → method must be overridden.
- `final` → method cannot be overridden.

Combining them is a logical contradiction and produces a compile-time error.

Q8. What is the difference between abstraction and encapsulation? Provide concrete examples.

Aspect	Abstraction	Encapsulation
Purpose	Hide implementation complexity	Hide internal data
Concept	Design-level (what to expose)	Implementation-level (how to protect)
Mechanism	abstract classes, interfaces	private fields + access methods
Question answered	What does it do?	How is it stored?
Example	Database driver interface	BankAccount with private balance

```
// ABSTRACTION (what)

interface Database {

    void connect();

    void query(String sql);

}

// ENCAPSULATION (how – internal state hidden)

class MySQLDatabase implements Database {

    private Connection conn;           // Hidden state

    private String url;                 // Hidden state

    public void connect() {

        // implementation hidden from caller

    }

    public void query(String sql) {

        // ...

    }

}
```

Q9. What happens if a subclass does not override all abstract methods?

✓ PERFECT ANSWER

The subclass itself must be declared abstract. It cannot be instantiated until ALL inherited abstract methods are implemented at some level of the hierarchy.

```
abstract class Shape {  
    abstract double area();  
    abstract double perimeter();  
}  
  
abstract class Polygon extends Shape {  
    // Doesn't implement either method — must be abstract itself  
}  
  
class Triangle extends Polygon {  
    @Override double area()      { return 0; }  
    @Override double perimeter() { return 0; }  
}
```

Q10. Give real-world examples of abstraction in the JDK.

✓ PERFECT ANSWER

- 82. List, Set, Map interfaces — define WHAT collections do; ArrayList, HashSet, HashMap define HOW.
- 83. InputStream / OutputStream abstract classes — abstract over file, socket, byte array streams.
- 84. AbstractList, AbstractMap, AbstractSet — partial implementations to ease writing custom collections.
- 85. HttpServlet, AbstractHandler — frameworks use abstract classes for extension points.
- 86. Runnable, Callable — abstract over executable tasks.

Q11. What is partial abstraction vs full abstraction?

✓ PERFECT ANSWER

Full abstraction (100%): All methods are abstract. Achieved via interfaces (pre-Java 8) — only method signatures, no implementation.

Partial abstraction (0–100%): Mix of abstract and concrete methods. Achieved via abstract classes. Allows shared implementation + extension points.

Java 8+: Interfaces can have default and static methods, blurring the line. The strict 'interface = 100% abstraction' rule no longer holds.

Q12. How does abstraction support loose coupling?

✓ PERFECT ANSWER

By programming to abstractions (interfaces or abstract classes) rather than concrete implementations, your code depends on stable contracts. Implementations can be swapped without changing dependent code — the foundation of dependency injection and the SOLID principles (specifically the Dependency Inversion Principle).

```
// Tightly coupled – bad
class OrderService {
    MySQLDatabase db = new MySQLDatabase();    // Hardcoded
}

// Loosely coupled – good (depends on abstraction)
class OrderService {
    private Database db;

    public OrderService(Database db) {          // Inject any implementation
        this.db = db;
    }
}

// At runtime, choose implementation:
Database db = new PostgresDatabase();
OrderService service = new OrderService(db);
```

**SUMMARY: Abstraction**

- Hide HOW; expose WHAT. Use abstract classes (0–100%) or interfaces (100% pre-Java 8).
- Abstract classes can have constructors, state, and concrete methods.
- Abstract methods cannot be private, static, or final.
- Programming to abstractions enables loose coupling and dependency injection.
- Java 8+ default methods on interfaces blur the abstract class / interface distinction.

TOPIC 2.5: ACCESS MODIFIERS ★★★★★**Q1. What are the access modifiers in Java? Explain visibility scope.**

✓ PERFECT ANSWER

Java has four access modifiers controlling member visibility:

Modifier	Same Class	Same Package	Subclass (diff pkg)	Other classes (diff pkg)
private	✓	✗	✗	✗
default (no modifier)	✓	✓	✗	✗
protected	✓	✓	✓	✗
public	✓	✓	✓	✓

```
package com.example;

public class Demo {
    private    int a;    // Only inside Demo
    int        b;        // Same package
    protected int c;    // Same package + subclasses anywhere
    public     int d;    // Everywhere
}
```



WHAT INTERVIEWER TESTS

- Knowing the visibility scope of each modifier exactly.
- Understanding the difference between default and protected.
- Which modifiers can be applied to top-level classes (only public or default).
- Implications for inheritance, encapsulation, and API design.

Q2. What is the difference between default and protected access?

✓ PERFECT ANSWER

default (package-private): Visible only within the same package. Subclasses in OTHER packages CANNOT access.

protected: Visible within the same package AND to subclasses in any package (only via inheritance, not via instance reference).

```
// Package com.a
package com.a;

public class A {
    protected int p;
    int          d;    // default
}

// Package com.b
package com.b;
import com.a.A;

class B extends A {
    void show() {
        System.out.println(p);    // OK – protected, accessed via
inheritance
        // System.out.println(d); // ERROR – default not visible
    }
}
```

Q3. What modifiers can be applied to top-level classes?

✓ PERFECT ANSWER

- public — accessible from any package.
- (no modifier / default) — accessible only within its package.
- final — cannot be subclassed.
- abstract — cannot be instantiated.
- sealed / non-sealed (Java 17+) — controlled inheritance.

NOT allowed on top-level classes: private, protected, static.

Q4. Can a private class exist? When and where?

✓ PERFECT ANSWER

YES — but only as a NESTED class (inside another class). Top-level classes cannot be private. Private nested classes are useful for implementation hiding within an enclosing class.

```
public class Outer {  
    private static class HelperImpl {  
        void helperMethod() { }  
    }  
  
    public void doWork() {  
        HelperImpl h = new HelperImpl();    // OK inside Outer  
        h.helperMethod();  
    }  
}  
  
// Outer.HelperImpl h = ...;                // ERROR outside Outer
```

Q5. Why are constants typically public, static, and final?

✓ PERFECT ANSWER

- **public** — accessible to all clients.
- **static** — single shared copy, no instance needed.
- **final** — immutable value; safer and JIT-optimized as compile-time constant.

Convention: ALL_CAPS_WITH_UNDERSCORES for the identifier.

```
public class HttpConstants {  
    public static final int OK = 200;  
    public static final int NOT_FOUND = 404;  
    public static final String DEFAULT_ENCODING = "UTF-8";  
}
```

Q6. What is the most restrictive access modifier? When should you use it?

✓ PERFECT ANSWER

'private' is the most restrictive — visible only within the same class. Use it as the DEFAULT for all class members; widen access only when necessary. This is the principle of least privilege.

Always make private:

- All instance fields (encapsulation).
- Helper methods used only internally.
- Constructors of utility classes (force static usage).

Q7. Can the overriding method have less restrictive access than the parent?

✓ PERFECT ANSWER

YES — an overriding method CAN have WIDER (less restrictive) access, but NEVER narrower. This preserves Liskov substitution: any client expecting the parent's API must work with subclass instances.

```
class Parent {  
    protected void show() { }  
}  
  
class Child extends Parent {  
    @Override  
    public void show() { }      // WIDER access – OK  
  
    // @Override  
    // private void show() { } // ERROR – narrower than parent  
}
```

Q8. Can an interface method be private? (Java 9+)

✓ PERFECT ANSWER

YES, since Java 9. Private methods in interfaces are used to share common code between default methods within the same interface. They are NOT visible to implementing classes.

Private interface methods can be: static or instance, but always non-abstract.

```
interface MyInterface {  
    default void method1() {  
        helper();          // OK – private method in same interface  
    }  
  
    default void method2() {  
        helper();  
    }  
  
    private void helper() {          // Java 9+  
        System.out.println("Shared logic");  
    }  
}
```

Q9. What is module-level access (Java 9+ modules)?

✓ PERFECT ANSWER

Java 9 introduced the Module System. A module-info.java file declares which packages are exported. A 'public' class is only visible to other modules if its package is explicitly EXPORTED. This adds a layer above the four traditional access modifiers.

Result: 'public' inside a non-exported package is effectively module-private.

```
module com.example.app {  
    exports com.example.app.api;  
  
    // com.example.app.internal NOT exported – invisible to other modules  
}
```

Q10. Best practices for choosing access modifiers.

✓ PERFECT ANSWER

87. Start with private. Widen only when justified.
88. Fields → always private (except constants).
89. Helper methods → private.
90. Methods called by subclasses → protected.
91. Methods called by external clients → public.
92. Public API surface should be as small as possible — easier to maintain backward compatibility.



SUMMARY: Access Modifiers

- private < default < protected < public — widest visibility goes right.
- Top-level classes can be only public or default.
- protected = same package + subclasses (via inheritance).
- Overriding methods may widen access, never narrow.
- Java 9+ modules add another layer of access control above the four modifiers.

PART 3: ADVANCED LEVEL

Advanced OOP topics dominate senior-level interviews. Interface vs Abstract Class is asked in 90%+ of Java backend interviews. Composition vs Inheritance is critical for design discussions.

TOPIC 3.1: INTERFACE ★★★★★

Q1. What is an interface? Why do we use it?

✓ PERFECT ANSWER

An interface is a contract that defines WHAT a class must do, but not HOW. It contains abstract methods (implicitly public abstract) and constants (implicitly public static final). Before Java 8, it provided 100% abstraction.

Why use interfaces:

- Achieve 100% abstraction (pre-Java 8).
- Support multiple inheritance of TYPE in Java.
- Loose coupling — clients depend on contract, not implementation.
- Enable polymorphism across unrelated classes.
- Foundation for design patterns (Strategy, Observer, Factory).
- Enable lambdas via functional interfaces (Java 8+).

```
interface Drawable {  
    int DEFAULT_SIZE = 10;           // implicitly public static final  
  
    void draw();                     // implicitly public abstract  
}  
  
class Circle implements Drawable {  
    public void draw() {              // must be public  
        System.out.println("Drawing circle");  
    }  
}  
  
class Rectangle implements Drawable {  
    public void draw() {  
        System.out.println("Drawing rectangle");  
    }  
}  
  
Drawable d = new Circle();           // Programming to interface  
d.draw();
```



WHAT INTERVIEWER TESTS

- Knowing all the implicit modifiers.
- Evolution of interfaces from Java 8 to Java 17.
- Multiple inheritance and diamond problem resolution.
- When to choose interface over abstract class.



FOLLOW-UP QUESTIONS

- Can an interface extend another interface? → Yes, even multiple.
- Can an interface have a constructor? → No, never.
- Are interface methods implicitly final? → No, they're abstract by default.
- What's a functional interface? → Interface with exactly ONE abstract method.

⚠ COMMON MISTAKES

- **✗** Saying 'interface = abstract class without state' — interfaces are more nuanced post-Java 8.
- **✗** Forgetting that implementing methods must be public (cannot be reduced).
- **✗** Thinking interface methods are static — only static methods declared with 'static' keyword are.

💡 HOW TO SPEAK ALOUD

"An interface is a contract — it declares what a class must do, not how. Implementing classes provide the actual behavior. Unlike inheritance, a single class can implement multiple interfaces, giving Java its form of multiple inheritance of TYPE. Since Java 8, interfaces can also have default and static methods with bodies, blurring the line with abstract classes — but interfaces still cannot maintain state."

Q2. What changed in interfaces from Java 8 to Java 17?

Feature	≤ Java 7	Java 8	Java 9+
Abstract methods	✓	✓	✓
Constants (public static final)	✓	✓	✓
default methods (with body)	✗	✓	✓
static methods	✗	✓	✓
private methods	✗	✗	✓
private static methods	✗	✗	✓

```
interface MyInterface {  
    int CONST = 10;                                // public static final  
  
    void abstractMethod();                          // public abstract  
  
    default void defaultMethod() {                  // Java 8  
        helper();  
    }  
  
    static void staticMethod() {                    // Java 8  
        System.out.println("Static");  
    }  
  
    private void helper() {                          // Java 9+  
        System.out.println("Helper");  
    }  
}
```

Q3. What is a default method? Why was it added?

✓ PERFECT ANSWER

A default method is a method in an interface with a body, prefixed with the 'default' keyword. Implementing classes inherit it automatically unless they override it.

Reasons for introduction (Java 8):

93. Backward compatibility — add new methods (e.g., `forEach` on `Iterable`) to existing interfaces without breaking implementations.
94. Enable Stream API integration on `Collection`.
95. Reduce boilerplate for utility-like methods.
96. Bridge the gap between interfaces and abstract classes.

```
interface Vehicle {  
    void start();  
  
    default void honk() {  
        System.out.println("Beep!");  
    }  
}  
  
class Car implements Vehicle {  
    public void start() { }  
    // honk() inherited – no need to implement  
}  
  
class Truck implements Vehicle {  
    public void start() { }  
    @Override  
    public void honk() {  
        System.out.println("HOOONK!");    // Override default  
    }  
}
```

Q4. What is the diamond problem with default methods? How is it resolved?

✓ PERFECT ANSWER

If a class implements two interfaces with conflicting default methods (same signature), the compiler reports an error. The class **MUST** resolve the ambiguity by overriding the method explicitly.

Resolution rules:

97. Class methods > Interface default methods (concrete class method always wins).
98. More specific interface > Less specific (subinterface default wins over superinterface).
99. Otherwise, the implementing class **MUST** override and choose explicitly.

```
interface A {  
    default void hello() { System.out.println("A"); }  
}  
  
interface B {  
    default void hello() { System.out.println("B"); }  
}  
  
class C implements A, B {  
    @Override  
    public void hello() {  
        A.super.hello();           // Explicit choice  
        B.super.hello();           // Or call both  
        System.out.println("C's own");  
    }  
}
```

Q5. What is a marker interface? Give examples.

✓ PERFECT ANSWER

A marker interface is an interface with NO methods. It exists solely to TAG a class as having a property, which the JVM or frameworks check via 'instanceof'.

Built-in markers:

- Serializable — class can be serialized.
- Cloneable — class supports clone().
- RandomAccess — list supports O(1) index access.
- Remote — RMI-accessible object.

Modern alternative: Annotations (e.g., @Entity, @Service) — more expressive than markers.

```
import java.io.Serializable;

class Employee implements Serializable {    // Marker tag

    int id;

    String name;

}

ObjectOutputStream out = new ObjectOutputStream(new
FileOutputStream("e.ser"));

out.writeObject(new Employee());           // Works because of marker

// Without Serializable → NotSerializableException
```

Q6. What is a functional interface? List Java's built-in functional interfaces.

✓ PERFECT ANSWER

A functional interface has exactly ONE abstract method (SAM — Single Abstract Method). It can have any number of default and static methods. The `@FunctionalInterface` annotation enforces this at compile time.

Built-in functional interfaces (java.util.function):

Interface	Method	Description
Function<T, R>	R apply(T t)	Transform $T \rightarrow R$
Predicate<T>	boolean test(T t)	Test condition
Consumer<T>	void accept(T t)	Consume value
Supplier<T>	T get()	Produce value
BiFunction<T,U,R>	R apply(T t, U u)	Two inputs $\rightarrow R$
UnaryOperator<T>	T apply(T t)	$T \rightarrow T$
BinaryOperator<T>	T apply(T t, T t)	$(T, T) \rightarrow T$
Runnable	void run()	No input, no output
Callable<V>	V call() throws Exception	Returns V, can throw
Comparator<T>	int compare(T, T)	Comparison logic

```
@FunctionalInterface

interface Calculator {

    int operate(int a, int b);

    // int doubleOp(int a, int b);    // ERROR — second abstract method
}

Calculator add = (a, b) -> a + b;
Calculator mul = (a, b) -> a * b;

System.out.println(add.operate(3, 4));    // 7
System.out.println(mul.operate(3, 4));    // 12

// Using built-in functional interfaces
Function<String, Integer> length = String::length;
Predicate<Integer> isEven = n -> n % 2 == 0;
Supplier<LocalDate> today = LocalDate::now;
Consumer<String> printer = System.out::println;
```

Q7. Can an interface extend a class? Can a class extend an interface?

✓ PERFECT ANSWER

Interface extend class: NO. Interfaces can extend only other interfaces (and they can extend multiple at once).

Class extend interface: NO — but a class can IMPLEMENT interfaces. The keyword is 'implements', not 'extends'.

```

interface A { }

interface B { }

interface C extends A, B { }           // Multi-extends — OK

class D implements A, B { }           // implements — OK

// class E extends A { }              // ERROR

// interface F extends Object { }     // ERROR — Object is a class

```

Q8. Why are interface methods implicitly public?

✓ PERFECT ANSWER

Interfaces define a public CONTRACT. If interface methods were package-private or protected, implementing classes outside the package couldn't satisfy the contract — defeating the purpose of polymorphic dispatch through the interface reference.

Corollary: Overriding methods in implementing classes MUST be declared public (cannot reduce visibility — Liskov substitution).

Q9. Comparable vs Comparator — when to use which?

Feature	Comparable<T>	Comparator<T>
Package	java.lang	java.util
Method	compareTo(T o)	compare(T o1, T o2)
Location	Inside the class	External, separate
Purpose	Natural ordering	Custom ordering(s)
Modify source?	Yes — must implement	No — independent
Multiple criteria	Single ordering only	Many independent orderings
Functional interface?	No (different SAM)	Yes — can use lambdas

```
// Natural ordering – Comparable

class Employee implements Comparable<Employee> {

    int id;

    @Override

    public int compareTo(Employee e) {

        return Integer.compare(this.id, e.id);

    }

}

// Custom orderings – Comparator

Comparator<Employee> byName          = Comparator.comparing(e -> e.name);

Comparator<Employee> bySalaryDesc =
    Comparator.comparingDouble(Employee::getSalary).reversed();

Comparator<Employee> chained        = byName.thenComparing(bySalaryDesc);

list.sort(chained);
```

Q10. Can an interface have a main method?

✓ PERFECT ANSWER

YES, since Java 8 — because main is static and interfaces can have static methods. You can run the interface directly: `java InterfaceName`.

```
interface Runnable2 {

    static void main(String[] args) {

        System.out.println("Interface main running");

    }

}

// $ java Runnable2 → "Interface main running"
```



SUMMARY: Interface

- Defines a contract; methods implicitly public abstract; fields implicitly public static final.
- Java 8 added default + static methods; Java 9 added private methods.
- Supports multiple inheritance; diamond resolved via explicit override.
- Functional interface = SAM; powers lambdas and Stream API.
- Use interface for contracts among unrelated classes.

TOPIC 3.2: ABSTRACT CLASS

Q1. What is an abstract class? When should you use one?

✓ PERFECT ANSWER

An abstract class is a class declared with 'abstract' keyword. It cannot be instantiated directly. It may contain both abstract methods (without body) and concrete methods (with body). It is meant to be subclassed.

Use abstract class when:

100. Related classes share substantial common code.
101. You want partial implementation with hooks (Template Method pattern).
102. You need shared state (instance fields).
103. You need non-public members in the contract (protected, default).
104. Strong IS-A hierarchy is required.

```
abstract class Shape {  
    protected String color;                // Shared state  
  
    public Shape(String color) {            // Constructor allowed  
        this.color = color;  
    }  
  
    abstract double area();                 // Subclass must implement  
    abstract double perimeter();  
  
    void describe() {                       // Shared behavior  
        System.out.println(color + " shape: area=" + area());  
    }  
}  
  
class Circle extends Shape {  
    double radius;  
    Circle(String c, double r) {  
        super(c);  
        this.radius = r;  
    }  
    @Override double area() { return Math.PI * radius * radius; }  
    @Override double perimeter() { return 2 * Math.PI * radius; }  
}
```



WHAT INTERVIEWER TESTS

- Understanding the dual nature: abstract + concrete methods coexist.
- Awareness that abstract classes can have constructors and state.
- Knowledge of Template Method pattern.
- When to choose abstract over interface.

+ FOLLOW-UP QUESTIONS

- Can an abstract class be instantiated? → No, never directly.
- Can it have a constructor? → Yes — invoked via `super()` from subclass.
- Can it have all concrete methods? → Yes — useful to prevent direct instantiation.
- Can it have a main method? → Yes — main is static, allowed anywhere.

⚠ COMMON MISTAKES

- ❌ Saying 'abstract class cannot have constructors'.
- ❌ Confusing abstract method (no body) with method that has empty body `{ }`.
- ❌ Forgetting that subclass **MUST** implement **ALL** abstract methods or itself be abstract.

Q2. Can an abstract class have a constructor? Why?

✓ PERFECT ANSWER

YES — and they usually do. Although the abstract class cannot be instantiated directly, its constructors are invoked via `super()` when a concrete subclass is constructed. They initialize the inherited state.

```
abstract class Animal {  
    protected String name;  
  
    Animal(String name) {                // Constructor — initializes  
shared state  
        this.name = name;  
    }  
  
    abstract void sound();  
}  
  
class Dog extends Animal {  
    Dog(String name) {  
        super(name);                    // Calls Animal's constructor  
    }  
    void sound() { System.out.println(name + " barks"); }  
}  
  
new Dog("Buddy").sound();                // "Buddy barks"
```

Q3. Can abstract methods be private, static, or final?

✓ PERFECT ANSWER

- private: NO — private methods aren't visible to subclasses, defeating override.
- static: NO — static methods belong to class and cannot be overridden (only hidden).
- final: NO — final prevents override; abstract REQUIRES override. Contradiction.

All three combinations are compile-time errors.

```
abstract class Demo {  
    // abstract private void m1();           // ERROR  
    // abstract static void m2();           // ERROR  
    // abstract final void m3();           // ERROR  
    abstract void m4();                     // OK (package-private)  
    public abstract void m5();             // OK  
    protected abstract void m6();         // OK  
}
```

Q4. Can an abstract class implement an interface without implementing all methods?

✓ PERFECT ANSWER

YES — and this is a common pattern. Unimplemented interface methods remain abstract in the abstract class, deferring implementation to concrete subclasses. The JDK uses this extensively (AbstractList, AbstractMap, etc.).

```
interface Vehicle {  
    void start();  
    void stop();  
    void accelerate();  
}  
  
abstract class AbstractVehicle implements Vehicle {  
    public void start() { System.out.println("Common start"); }  
    public void stop() { System.out.println("Common stop"); }  
    // accelerate() unimplemented – remains abstract  
}  
  
class Car extends AbstractVehicle {  
    public void accelerate() {  
        System.out.println("Car accelerating");  
    }  
}
```

Q5. What is the Template Method design pattern?

✓ PERFECT ANSWER

Template Method is a behavioral design pattern that defines the skeleton of an algorithm in a method (typically final), deferring some steps to subclasses. Subclasses can redefine specific steps WITHOUT changing the algorithm's structure.

Implementation: Abstract class with final 'template method' calling abstract 'hook methods' implemented by subclasses.

```
abstract class Beverage {  
    final void prepare() {                // Template – non-overridable  
        boilWater();  
        brew();                          // Hook  
        pourInCup();  
        addCondiments();                // Hook  
    }  
  
    private void boilWater() { System.out.println("Boiling water"); }  
    private void pourInCup() { System.out.println("Pouring in cup"); }  
  
    abstract void brew();                // Subclass customization  
    abstract void addCondiments();  
}  
  
class Tea extends Beverage {  
    void brew() { System.out.println("Steeping tea bag"); }  
    void addCondiments() { System.out.println("Adding lemon"); }  
}  
  
class Coffee extends Beverage {  
    void brew() { System.out.println("Brewing coffee"); }  
    void addCondiments() { System.out.println("Adding sugar and milk"); }  
}
```

Q6. Can abstract class have final methods? What's the use?

✓ PERFECT ANSWER

YES — and this combination is powerful. A final method enforces non-overridable behavior, while abstract methods in the same class force subclass customization. This is the Template Method pattern's foundation.

Q7. What happens if a subclass doesn't implement all abstract methods?

✓ PERFECT ANSWER

The subclass itself **MUST** be declared abstract. It cannot be instantiated until **ALL** inherited abstract methods are implemented at some level of the hierarchy.

```
abstract class Shape {  
    abstract double area();  
    abstract double perimeter();  
}  
  
abstract class Polygon extends Shape {           // Must be abstract — no impls  
    int sides;  
}  
  
class Triangle extends Polygon {                 // Concrete — must implement  
    both  
    @Override double area()      { return 0; }  
    @Override double perimeter() { return 0; }  
}
```

Q8. Can abstract class have static methods, private methods, and main method?

✓ PERFECT ANSWER

- Static methods: YES — class-level utility methods, accessible via class name.
- Private methods: YES — helpers within the abstract class, invisible to subclasses.
- Main method: YES — main is just a static method; you can run an abstract class directly.

```
abstract class Demo {  
    abstract void abstractMethod();  
  
    static void util() { // OK  
        System.out.println("Utility");  
    }  
  
    private void helper() { // OK  
        System.out.println("Helper");  
    }  
  
    public static void main(String[] args) { // OK  
        System.out.println("Main");  
        Demo.util();  
    }  
}  
  
// $ java Demo → "Main\nUtility"
```

Q9. Real-world JDK examples of abstract classes.

✓ PERFECT ANSWER

- 105. AbstractList, AbstractMap, AbstractSet — partial implementations for Collections framework.
- 106. InputStream, OutputStream — abstract base for byte streams (FileInputStream, ByteArrayInputStream extend).
- 107. Reader, Writer — abstract base for character streams.
- 108. HttpServlet — base for HTTP servlets; defines doGet, doPost as hook methods.
- 109. Number — abstract parent of Integer, Double, Long, etc.
- 110. AbstractExecutorService — partial implementation for ExecutorService.

Q10. When would you choose an abstract class over an interface with default methods?

✓ PERFECT ANSWER

- 111. Need to maintain INSTANCE STATE (interfaces cannot have instance fields).
- 112. Need a CONSTRUCTOR for shared initialization.
- 113. Need PROTECTED or package-private members (interface methods are public).
- 114. Need a FINAL method (interfaces cannot).
- 115. Want to enforce a strong IS-A hierarchy with shared identity.
- 116. Need to share non-trivial implementation with controlled customization (Template Method).



SUMMARY: Abstract Class

- Cannot be instantiated; designed for inheritance.
- Mix of abstract and concrete methods; constructors allowed.
- Can have state, final methods, protected members — unlike interfaces.
- Template Method pattern: final template + abstract hooks.
- Choose for code reuse among RELATED classes.

TOPIC 3.3: INTERFACE vs ABSTRACT CLASS ★★★★★



VVI: MOST-ASKED OOP QUESTION IN BACKEND INTERVIEWS

This is THE most-asked OOP question — appears in ~90% of Java backend interviews. Master the comparison table, decision framework, and at least two real-world scenarios for each choice.

Q1. Complete comparison: Interface vs Abstract Class

Aspect	Interface	Abstract Class
Keyword	interface	abstract class
Methods	abstract + default + static + private	abstract + concrete (any modifier)
Fields	public static final ONLY	Any: instance, static, final, private
Constructor	NOT allowed	Allowed
State (instance vars)	Cannot maintain	Can maintain
Multiple inheritance	YES — implements multiple	NO — extends one only
Instantiation	Cannot	Cannot directly
Access modifiers	Methods implicitly public	Any modifier
Final methods	Cannot have	Can have
Inheritance type	CAN-DO (capability)	IS-A (substance)
When to use	Contract among unrelated classes	Shared code among related classes

Q2. When should you choose an interface over an abstract class?

✓ PERFECT ANSWER

- 117. Multiple inheritance of TYPE is needed.
- 118. Unrelated classes share a capability (e.g., Comparable, Serializable).
- 119. You want pure contract with no implementation.
- 120. You want to enable lambdas (functional interface).
- 121. Loose coupling and easy mocking for testing.

```
// Unrelated classes share a CAPABILITY → interface  
  
interface Printable { void print(); }  
  
  
class Invoice implements Printable { public void print() { } }  
class Photo implements Printable { public void print() { } }  
class Report implements Printable { public void print() { } }  
  
// Invoice, Photo, Report unrelated but all CAN print
```

Q3. When should you choose an abstract class over an interface?

✓ PERFECT ANSWER

- 122. Related classes share substantial common implementation.
- 123. You need to maintain state (instance fields).
- 124. You need a constructor for shared initialization.
- 125. Non-public members are needed in the contract.
- 126. Template Method pattern with controlled hooks.
- 127. Strong IS-A hierarchy.

```
// Related classes share IMPLEMENTATION → abstract class

abstract class HttpServletBase {

    protected void log(String msg) {                // Shared

        System.out.println("LOG: " + msg);

    }

    abstract void handleRequest(HttpServletRequest req);

}

class GetServlet extends HttpServletBase {

    void handleRequest(HttpServletRequest req) {

        log("Handling GET");

    }

}
```

Q4. Scenario: Design a payment system with CreditCard, DebitCard, UPI.

✓ PERFECT ANSWER

Choice: Interface — PaymentMethod.

Reasoning:

- All three are payment methods, but implementations are entirely different (card networks, bank APIs, UPI rails).
- No common code to share.
- Loose coupling needed — Checkout shouldn't know specific type.
- Easy to add new types (PayPal, Crypto, Wallet).
- Enables Strategy pattern.

```
interface PaymentMethod {

    boolean authorize(double amount);

    boolean capture(double amount);

    void refund(double amount);

}

class CreditCardPayment implements PaymentMethod { /* card network */ }

class DebitCardPayment implements PaymentMethod { /* bank API */ }

class UpiPayment implements PaymentMethod { /* NPCI */ }

class Checkout {

    private PaymentMethod method;

    Checkout(PaymentMethod m) { this.method = m; }

    void process(double amount) {

        if (method.authorize(amount)) method.capture(amount);

    }

}
```

Q5. Scenario: Design a logger system with FileLogger, ConsoleLogger, DBLogger.

✓ PERFECT ANSWER

Choice: Abstract class — AbstractLogger.

Reasoning:

- All loggers share substantial common logic: timestamp formatting, level filtering, message structure.
- Only the WRITE step differs — perfect for Template Method.
- Shared state (log level, format pattern) is natural in an abstract class.
- Strong IS-A relationship — they are all loggers.

```
abstract class AbstractLogger {  
    protected LogLevel level = LogLevel.INFO;  
  
    public final void log(LogLevel lvl, String msg) {    // Template  
        if (lvl.compareTo(level) < 0) return;  
        write(format(lvl, msg));  
    }  
  
    private String format(LogLevel l, String m) {  
        return Instant.now() + " [" + l + "] " + m;  
    }  
  
    protected abstract void write(String formatted);    // Hook  
}  
  
class FileLogger extends AbstractLogger {  
    protected void write(String s) { /* write to file */ }  
}  
  
class ConsoleLogger extends AbstractLogger {  
    protected void write(String s) { System.out.println(s); }  
}
```

Q6. Can a class implement multiple interfaces AND extend a class simultaneously?

✓ PERFECT ANSWER

YES — this is the standard pattern: extend ONE class and implement MANY interfaces.

```
class Service extends AbstractService

    implements Runnable, AutoCloseable, Serializable {

    // From AbstractService: inherited state and methods

    // From Runnable:    must implement run()

    // From AutoCloseable: must implement close()

    // From Serializable: marker — nothing to implement

}
```

Q7. JDK Collections — how are both used together?

✓ PERFECT ANSWER

- Interfaces (contracts): Collection, List, Set, Map, Queue, Deque, Iterator.
- Abstract classes (skeletal): AbstractCollection, AbstractList, AbstractMap, AbstractSet.
- Concrete classes: ArrayList extends AbstractList implements List; HashMap extends AbstractMap implements Map.

This layered design gives reusable implementation via abstract classes while exposing the public contract via interfaces — best of both worlds.

Q8. Decision Framework — How to choose.

✓ PERFECT ANSWER

Step 1:

Is there a strong IS-A relationship AND substantial shared code?

- YES → abstract class candidate.
- NO → interface candidate.

Step 2:

Do you need state, constructors, or non-public members?

- YES → abstract class.
- NO → interface.

Step 3:

Will implementations be unrelated (numbers, strings, dates all sortable)?

- YES → interface.

Step 4:

Modern Java best practice: prefer interface + default methods unless you need abstract class features.



SUMMARY: Interface vs Abstract Class

- Interface = CAN-DO; Abstract class = IS-A.
- Interface for unrelated classes sharing capability.
- Abstract class for related classes sharing code.
- Need state, constructor, protected? → abstract class.
- Default in modern Java: interface + default methods.

TOPIC 3.4: COMPOSITION vs INHERITANCE

Q1. What is composition? How does it differ from inheritance?

✓ PERFECT ANSWER

Composition is a HAS-A relationship where one class contains a reference to another class as a field, delegating work to it. Inheritance is an IS-A relationship using extends.

Inheritance vs Composition:

- Inheritance: Class B IS-A class A — uses 'extends'.
- Composition: Class B HAS-A class A — uses field reference + delegation.

```
// WRONG — Car is NOT an Engine
class Engine { void start() { } }
class Car extends Engine { }           // Misuse — Car is not an Engine

// CORRECT — Car HAS-A Engine
class Engine { void start() { } }
class Car {
    private Engine engine = new Engine();    // Composition
    void start() { engine.start(); }         // Delegation
}
```



REAL-WORLD ANALOGY

"Inheritance: 'A Dog IS-A Animal' — Dog inherits traits. Composition: 'A Car HAS-A Engine' — Car uses an Engine but is not one. Genetic inheritance vs assembling parts."



WHAT INTERVIEWER TESTS

- Understanding IS-A vs HAS-A semantics.
- Knowledge of 'favor composition over inheritance' (Effective Java).
- Identifying misuse of inheritance.
- Awareness of patterns built on composition (Strategy, Decorator).

Q2. Why is composition preferred over inheritance?

✓ PERFECT ANSWER

- 128. Greater flexibility — behavior can change at runtime by swapping the composed object.
- 129. Looser coupling — depends only on public API.
- 130. Better testability — mock the dependency.
- 131. Avoids the fragile base class problem.
- 132. No diamond problem — many objects can be composed unambiguously.
- 133. Encapsulation preserved — inheritance can leak parent's internals.

Q3. What is the fragile base class problem?

✓ PERFECT ANSWER

The fragile base class problem occurs when modifications to a parent class break subclasses, even though subclass code did not change. This happens because inheritance creates dependency on parent's internal implementation, not just its public API.

```
class Counter {  
    int count = 0;  
    void inc()      { count++; }  
    void incBy(int n) { count += n; }  
}  
  
class TrackedCounter extends Counter {  
    int totalInc = 0;  
    @Override void inc()      { totalInc++; super.inc(); }  
    @Override void incBy(int n) { totalInc += n; super.incBy(n); }  
}  
  
// Later, Counter refactors:  
// void incBy(int n) { for (int i = 0; i < n; i++) inc(); }  
// Now TrackedCounter double-counts (incBy increments totalInc by n,  
// then super.incBy calls inc() n times, which also increments totalInc).
```

Q4. Demonstrate Strategy pattern using composition.

✓ PERFECT ANSWER

Strategy pattern uses composition to swap algorithms at runtime — a textbook example of 'favor composition'.

```
interface SortStrategy {
    void sort(int[] arr);
}

class QuickSort implements SortStrategy {
    public void sort(int[] arr) { /* quicksort */ }
}

class MergeSort implements SortStrategy {
    public void sort(int[] arr) { /* mergesort */ }
}

class Sorter {
    private SortStrategy strategy;                // Composition

    public Sorter(SortStrategy s) { this.strategy = s; }

    public void setStrategy(SortStrategy s) {      // Swap at runtime
        this.strategy = s;
    }

    public void run(int[] arr) { strategy.sort(arr); }
}

Sorter s = new Sorter(new QuickSort());
s.run(arr);
s.setStrategy(new MergeSort());                 // Runtime behavior
change
s.run(arr);
```

Q5. What is delegation? How is it related to composition?

✓ PERFECT ANSWER

Delegation is forwarding a method call from one object to another (typically a composed field). Composition + delegation often replaces inheritance and is the foundation of Decorator, Proxy, and Strategy patterns.

Q6. Is inheritance ever the right choice over composition?

✓ PERFECT ANSWER

YES — when a true IS-A relationship exists and the parent's API is stable. Inheritance is appropriate when:

- Subclasses are conceptually a kind of parent (Square IS-A Shape).
- You want polymorphic substitution (LSP).
- Parent class is DESIGNED for inheritance (hooks documented, members protected).
- You need to expose protected members to subclasses.

Bad uses: pure code reuse, when composition would work equally well, extending concrete classes with public methods you don't want to inherit.

Q7. Demonstrate the Decorator pattern (composition-based).

✓ PERFECT ANSWER

Decorator dynamically adds behavior by wrapping objects. The decorator implements the same interface as the wrapped object and forwards calls, adding behavior before/after. The JDK uses this in `BufferedReader` → `InputStreamReader` → `FileInputStream`.

```
interface Coffee {
    double cost();
    String description();
}

class SimpleCoffee implements Coffee {
    public double cost() { return 5.0; }
    public String description() { return "Coffee"; }
}

abstract class CoffeeDecorator implements Coffee {
    protected Coffee wrapped; // Composition
    CoffeeDecorator(Coffee c) { this.wrapped = c; }
}

class MilkDecorator extends CoffeeDecorator {
    MilkDecorator(Coffee c) { super(c); }
    public double cost() { return wrapped.cost() + 1.5; }
    public String description() { return wrapped.description() + " + Milk"; }
}

class SugarDecorator extends CoffeeDecorator {
    SugarDecorator(Coffee c) { super(c); }
    public double cost() { return wrapped.cost() + 0.5; }
    public String description() { return wrapped.description() + " + Sugar"; }
}

Coffee c = new SugarDecorator(new MilkDecorator(new SimpleCoffee()));
// "Coffee + Milk + Sugar", cost: 7.0
```

Q8. How does composition enable better unit testing?

✓ PERFECT ANSWER

Composed dependencies are typically injected via constructor (Dependency Injection). In tests, mocks/stubs replace real dependencies, isolating the class under test.

```
class OrderService {  
    private PaymentGateway gateway;  
    public OrderService(PaymentGateway gw) { this.gateway = gw; }  
    public boolean placeOrder(Order o) {  
        return gateway.charge(o.getTotal());  
    }  
}  
  
// Test  
PaymentGateway mock = Mockito.mock(PaymentGateway.class);  
when(mock.charge(anyDouble())).thenReturn(true);  
OrderService svc = new OrderService(mock);  
assert svc.placeOrder(testOrder);           // No real payment
```

Q9. What does 'Favor composition over inheritance' mean in practice?

✓ PERFECT ANSWER

This GoF / Effective Java principle says to start with composition by default. Use inheritance only when there's a genuine IS-A relationship AND the parent class is designed for extension.

Practical interpretation:

- Don't reach for 'extends' just to reuse code.
- Inheritance creates strong coupling; composition keeps flexibility.
- Modern frameworks (Spring, Guice) are built on composition + DI.
- Modern Java features (Records, sealed classes) push toward composition.

Q10. Decision checklist: composition or inheritance?

✓ PERFECT ANSWER

- 134. Is B genuinely a kind of A? (IS-A test). Unsure → composition.
- 135. Will the parent evolve independently? → composition.
- 136. Need to combine multiple behaviors? → composition (avoid diamond).
- 137. Need runtime polymorphism via DI? → composition.
- 138. Certain about hierarchy + parent designed for extension? → inheritance OK.
- 139. Default in modern Java: composition.



SUMMARY: Composition vs Inheritance

- Inheritance = IS-A; Composition = HAS-A.
- Favor composition over inheritance (Effective Java).
- Composition gives flexibility, loose coupling, testability.
- Patterns built on composition: Strategy, Decorator, Proxy, Adapter, Observer.
- Use inheritance for genuine IS-A and parent designed for extension.

TOPIC 3.5: AGGREGATION vs COMPOSITION

Q1. What is aggregation? How is it different from composition?

✓ PERFECT ANSWER

Both are HAS-A relationships but differ in OWNERSHIP and LIFECYCLE:

- Aggregation (weak HAS-A): The contained object exists independently of the container.
- Composition (strong HAS-A): The contained object cannot exist without the container — its lifecycle is tied.

```
// AGGREGATION – Professor exists independently
class Department {
    private List<Professor> professors;

    Department(List<Professor> ps) { this.professors = ps; }
}

// Professors live independently of Department

// COMPOSITION – Engine dies with Car
class Engine {
    Engine() { /* ... */ }
}

class Car {
    private final Engine engine = new Engine();    // Created inside
}

// When Car is GC'd, Engine has no other reference and is GC'd too
```

Q2. What is association? How does it relate to aggregation and composition?

✓ PERFECT ANSWER

Association is the most general HAS-A relationship — any time one object uses or references another. Aggregation and composition are SPECIALIZATIONS of association.

Hierarchy: Association $\supseteq$ Aggregation $\supseteq$ Composition (strongest).

Relationship	Type	Lifecycle	Example
Association	Uses	Independent	Doctor treats Patient
Aggregation	Has-A (weak)	Independent	School has Teachers
Composition	Has-A (strong)	Dependent	House has Rooms

Q3. Code example: showing all three relationships.

```
// ASSOCIATION – Doctor uses Patient (transient use)

class Doctor {

    void treat(Patient p) {                                // Just a method param

        // ... treat patient

    }

}

// AGGREGATION – School has Teachers (existing externally)

class School {

    private List<Teacher> teachers;

    School(List<Teacher> ts) {

        this.teachers = ts;                                // Pre-existing
Teachers

    }

}

// COMPOSITION – House contains Rooms (cannot exist without House)

class House {

    private final List<Room> rooms;

    House() {

        rooms = List.of(                                    // Created inside

            new Room("Kitchen"),

            new Room("Bedroom"),

            new Room("Bathroom")

        );

    }

}
```

Q4. How does UML represent these relationships?

✓ PERFECT ANSWER

- Association: A simple line between classes.
- Aggregation: A line with an OPEN diamond on the container side.
- Composition: A line with a FILLED diamond on the container side.
- Multiplicity (1, *, 0..1) marks each end to show cardinality.

Q5. Why is the lifecycle distinction important in design?

✓ PERFECT ANSWER

- 140. Memory management — composition keeps memory lifetime aligned; aggregation requires GC reasoning across multiple owners.
- 141. Data integrity — composed parts are guaranteed exclusive; aggregated parts may be shared and mutated externally.
- 142. Encapsulation — composed parts can be hidden; aggregated parts have external references.
- 143. Database modeling — composition typically maps to cascade delete; aggregation does not.



SUMMARY: Aggregation vs Composition

- Both are HAS-A; differ in ownership/lifecycle.
- Aggregation: parts outlive the whole (School-Teacher).
- Composition: parts die with the whole (House-Room).
- UML: open diamond = aggregation; filled diamond = composition.
- Design impact: cascade delete, encapsulation, memory management.

TOPIC 3.6: COUPLING & COHESION

Q1. What are coupling and cohesion? Why do they matter?

✓ PERFECT ANSWER

Coupling: Degree of interdependence between modules/classes. Low coupling is desired.

Cohesion: Degree to which elements within a module/class belong together. High cohesion is desired.

Goal: Low coupling + High cohesion = maintainable, testable, scalable code.

Q2. Types of coupling (from worst to best).

Type	Description	Quality
Content coupling	Module accesses another's internal data	Worst
Common coupling	Modules share global data	Very bad
External coupling	Modules share an external format/protocol	Bad
Control coupling	Module controls another's flow via flags	Bad
Stamp coupling	Shared composite data structure	Moderate
Data coupling	Modules share only primitive parameters	Good
No coupling	Modules independent	Best

Q3. Types of cohesion (from worst to best).

Type	Description	Quality
Coincidental	Random grouping	Worst
Logical	Similar but unrelated operations	Bad
Temporal	Grouped by execution time	Bad
Procedural	Sequential steps	Moderate
Communicational	Operations on the same data	Good
Sequential	Output of one is input of next	Better
Functional	Single well-defined task	Best

Q4. Example: tight coupling vs loose coupling.

```
// TIGHT COUPLING – hardcoded dependency

class OrderService {

    private MySQLDatabase db = new MySQLDatabase();    // Hardcoded

    void save(Order o) { db.insert(o); }

}

// Cannot test without real MySQL; cannot swap to PostgreSQL.

// LOOSE COUPLING – depend on abstraction, inject implementation

interface Database {

    void insert(Order o);

}

class OrderService {

    private Database db;                                // Abstraction

    public OrderService(Database db) { this.db = db; }

    void save(Order o) { db.insert(o); }

}

// At runtime:

OrderService s = new OrderService(new MySQLDatabase());

// or

OrderService s = new OrderService(new PostgresDatabase());

// or in test:

OrderService s = new OrderService(mock(Database.class));
```

Q5. How do SOLID principles relate to coupling and cohesion?

✓ PERFECT ANSWER

- S — Single Responsibility → drives high cohesion (one class, one reason to change).
- O — Open/Closed → reduces coupling (extend without modifying).
- L — Liskov Substitution → safer inheritance, less brittle coupling.
- I — Interface Segregation → reduces coupling to bloated interfaces.
- D — Dependency Inversion → depend on abstractions, not concretes — minimizes coupling.

Q6. How does Dependency Injection reduce coupling?

✓ PERFECT ANSWER

Dependency Injection (DI) inverts the control of dependency creation — instead of a class creating its dependencies, they are provided from outside (constructor, setter, or framework). This achieves Dependency Inversion (the 'D' in SOLID).

Benefits:

- Class depends on abstraction, not concrete.
- Test with mocks easily.
- Swap implementations without modifying client code.
- Powers Spring's IoC container.

Q7. What is high cohesion? Give an example.

✓ PERFECT ANSWER

High cohesion means all members of a class are related to a single, well-defined responsibility — they 'belong together'.

```
// LOW COHESION – class does too much

class UserManager {

    void registerUser(User u)      { }

    void sendEmail(String msg)     { }                // Unrelated

    void exportToCSV(List<User> us) { }                // Unrelated

    void calculateTax(double amt)  { }                // Unrelated

}

// HIGH COHESION – split by responsibility

class UserRegistrationService {

    void register(User u) { }

}

class EmailService {

    void send(String msg) { }

}

class UserExportService {

    void exportToCSV(List<User> us) { }

}
```

Q8. Practical impact of coupling and cohesion on code maintainability.

✓ PERFECT ANSWER

- 144.Easier to test — high cohesion means focused tests.
- 145.Easier to refactor — low coupling means isolated changes.
- 146.Easier to understand — single-responsibility classes are self-documenting.
- 147.Easier to reuse — cohesive units fit more contexts.
- 148.Easier to scale — independent modules can be deployed/evolved separately (microservices).



SUMMARY: Coupling & Cohesion

- Low coupling + High cohesion = maintainable code.
- Tight coupling = hardcoded dependencies; loose = depend on abstractions.
- DI is the primary tool for reducing coupling.
- SOLID principles directly drive low coupling and high cohesion.
- Functional cohesion (one task per class) is the gold standard.

PART 4: SCENARIO-BASED & DESIGN QUESTIONS

Scenario-based questions test your ability to apply OOP principles to real systems. Always begin by IDENTIFYING ENTITIES, then RELATIONSHIPS, then APPLY OOP CONCEPTS. Speak through your design — interviewers value reasoning over speed.

★ VVI: DESIGN INTERVIEW FRAMEWORK

Step 1:

Clarify requirements — ask about scope, scale, edge cases.

Step 2:

Identify entities (nouns) → classes.

Step 3:

Identify relationships (IS-A, HAS-A) → inheritance/composition.

Step 4:

Apply OOP principles — encapsulation, abstraction, polymorphism.

Step 5:

Discuss extension points — how to add features without breaking existing code (Open/Closed).

SCENARIO 1: Library Management System ★★★★★

Q1. Design a library management system using OOP principles.

✓ PERFECT ANSWER

Entities identified:

- Library (manages everything).
- Book (the item being lent).
- Member (the borrower).
- Librarian (a special type of Member or separate).
- Loan / BorrowRecord (transaction).

Relationships:

- Library HAS-A many Books (composition).
- Library HAS-A many Members (aggregation — Members exist independently).
- Member CAN borrow Books (association).
- Librarian IS-A Member with extra privileges.

OOP principles applied:

- Encapsulation: private fields with controlled access.
- Inheritance: Librarian extends Member.
- Abstraction: Borrowable interface for items.
- Polymorphism: Book, DVD, Magazine — all implement Borrowable.

```
interface Borrowable {

    boolean borrow(Member m);

    boolean returnItem();

    boolean isAvailable();

}

abstract class LibraryItem implements Borrowable {

    private String id;

    private String title;

    protected boolean available = true;

    protected Member borrower;

    public LibraryItem(String id, String title) {

        this.id = id; this.title = title;

    }

    @Override

    public boolean borrow(Member m) {

        if (!available) return false;

        available = false;

        borrower = m;

        return true;

    }

    @Override

    public boolean returnItem() {

        if (available) return false;

        available = true;

        borrower = null;

        return true;

    }

}
```



WHAT INTERVIEWER TESTS

- Ability to identify entities, relationships, and apply OOP.
- Use of interface (Borrowable) to enable polymorphism.
- Use of abstract class (LibraryItem) for shared logic.
- Encapsulation through private fields and controlled APIs.



FOLLOW-UP QUESTIONS

- How would you add reservations? → Reservation class, Borrowable.reserve() default method.
- How would you handle late fees? → Loan entity with dueDate, calculateFee().
- How would you search? → SearchService with strategy (by title, author, ISBN).
- How would you persist? → Repository pattern, separate Book from BookEntity.

SCENARIO 2: ATM System

Q2. Design an ATM system.

✓ PERFECT ANSWER

Entities: ATM, Card, Account, Bank, Transaction.

Relationships:

- ATM HAS-A CashDispenser (composition).
- Card HAS-A Account reference (aggregation).
- Account HAS-A Transaction history.
- Transaction is abstract; Withdraw, Deposit, Transfer extend it.

Design patterns used:

- State pattern — ATM states (Idle, CardInserted, Authenticated, TransactionInProgress).
- Command pattern — each Transaction is a command.
- Strategy pattern — different account types have different withdrawal rules.

```
enum TransactionType { WITHDRAW, DEPOSIT, TRANSFER, BALANCE_INQUIRY }

abstract class Transaction {
    protected Account account;
    protected double amount;
    protected LocalDateTime timestamp = LocalDateTime.now();

    public Transaction(Account acc, double amount) {
        this.account = acc; this.amount = amount;
    }

    abstract boolean execute();
}

class WithdrawTransaction extends Transaction {
    public WithdrawTransaction(Account a, double amount) { super(a, amount); }

    @Override
    public boolean execute() {
        return account.withdraw(amount);
    }
}

class DepositTransaction extends Transaction {
    public DepositTransaction(Account a, double amount) { super(a, amount); }

    @Override
    public boolean execute() {
        account.deposit(amount);
    }
}
```

SCENARIO 3: Vehicle Rental System

Q3. Design a vehicle rental system.

✓ PERFECT ANSWER

Entities: Vehicle (abstract), Car, Bike, Truck, Customer, Rental, Inventory.

OOP applied:

- Inheritance: Car, Bike, Truck extend Vehicle.
- Polymorphism: each vehicle type computes pricing differently.
- Composition: Rental HAS-A Vehicle and HAS-A Customer.
- Strategy: PricingStrategy varies per vehicle type and rental duration.

```
abstract class Vehicle {

    protected String registrationNumber;

    protected String model;

    protected boolean available = true;

    public Vehicle(String reg, String model) {

        this.registrationNumber = reg; this.model = model;

    }

    public abstract double getDailyRate();

    public double calculateRent(int days) {

        return getDailyRate() * days;

    }

}

class Car extends Vehicle {

    private int seats;

    public Car(String reg, String model, int seats) {

        super(reg, model); this.seats = seats;

    }

    @Override public double getDailyRate() { return 50.0 + seats * 10; }

}

class Bike extends Vehicle {

    private int cc;

    public Bike(String reg, String model, int cc) {

        super(reg, model); this.cc = cc;

    }

    @Override public double getDailyRate() { return 15.0 + cc * 0.05; }

}
```

SCENARIO 4: Employee Management System

Q4. Design an employee management system with different employee types and salary calculation.

✓ PERFECT ANSWER

Entities: Employee (abstract), FullTimeEmployee, PartTimeEmployee, Contractor, Department, Company.

OOP applied:

- Abstract class: Employee with abstract calculateSalary().
- Inheritance: Three types extending Employee.
- Polymorphism: Same method calculateSalary() behaves differently.
- Composition: Department HAS-A list of Employees.

```
abstract class Employee {  
    protected String id;  
    protected String name;  
    protected double baseSalary;  
  
    public Employee(String id, String name, double base) {  
        this.id = id; this.name = name; this.baseSalary = base;  
    }  
  
    public abstract double calculateMonthlySalary();  
  
    public void displayDetails() {  
        System.out.println(name + " - " + calculateMonthlySalary());  
    }  
}  
  
class FullTimeEmployee extends Employee {  
    private double bonus;  
    public FullTimeEmployee(String id, String n, double base, double bonus)  
    {  
        super(id, n, base); this.bonus = bonus;  
    }  
    @Override public double calculateMonthlySalary() {  
        return baseSalary + bonus;  
    }  
}  
  
class PartTimeEmployee extends Employee {  
    private int hoursWorked;  
    private double hourlyRate;  
    public PartTimeEmployee(String id, String n, int hrs, double rate) {
```

SCENARIO 5: Parking Lot System

Q5. Design a parking lot with multiple vehicle types and dynamic pricing.

✓ PERFECT ANSWER

Entities: ParkingLot, ParkingSpot (abstract), CarSpot, BikeSpot, TruckSpot, Vehicle (abstract), Ticket, PricingStrategy.

OOP applied:

- Inheritance for vehicle and spot types.
- Strategy pattern for pricing (hourly, daily, premium).
- Encapsulation for parking lot state.
- Factory pattern for creating spots.

```
enum VehicleSize { SMALL, MEDIUM, LARGE }

abstract class Vehicle {
    protected String licensePlate;
    protected VehicleSize size;
    public Vehicle(String plate, VehicleSize size) {
        this.licensePlate = plate; this.size = size;
    }
}

class Car extends Vehicle { public Car(String p) { super(p, VehicleSize.MEDIUM); } }
class Bike extends Vehicle { public Bike(String p) { super(p, VehicleSize.SMALL); } }
class Truck extends Vehicle{ public Truck(String p){ super(p, VehicleSize.LARGE); } }

abstract class ParkingSpot {
    protected int spotId;
    protected boolean occupied = false;
    protected Vehicle vehicle;
    public abstract boolean canFit(Vehicle v);

    public boolean park(Vehicle v) {
        if (occupied || !canFit(v)) return false;
        this.vehicle = v;
        this.occupied = true;
        return true;
    }

    public void release() { vehicle = null; occupied = false; }
}
```

SCENARIO 6: Notification System

Q6. Design a notification system supporting Email, SMS, and Push notifications.

✓ PERFECT ANSWER

Choice: Interface — Notifier — because the channels are unrelated implementations.

Patterns used:

- Strategy pattern for picking a channel.
- Observer pattern for fan-out notifications.
- Factory pattern for creating notifiers.

```
interface Notifier {  
    void send(String recipient, String message);  
}  
  
class EmailNotifier implements Notifier {  
    public void send(String to, String msg) {  
        System.out.println("Email to " + to + ": " + msg);  
        // SMTP integration...  
    }  
}  
  
class SmsNotifier implements Notifier {  
    public void send(String phone, String msg) {  
        System.out.println("SMS to " + phone + ": " + msg);  
        // Twilio/MSG91...  
    }  
}  
  
class PushNotifier implements Notifier {  
    public void send(String deviceToken, String msg) {  
        System.out.println("Push to " + deviceToken + ": " + msg);  
        // FCM/APNS...  
    }  
}  
  
class NotificationService {  
    private List<Notifier> channels = new ArrayList<>();  
  
    public void register(Notifier n) { channels.add(n); }  
  
    public void broadcast(String recipient, String message) {
```

SCENARIO 7: E-Commerce Order System ★★★★★

Q7. Design an e-commerce order system with cart, payment, and shipping.

✓ PERFECT ANSWER

Entities: Product, Cart, CartItem, Order, Customer, PaymentMethod, ShippingMethod.

Patterns:

- Strategy: PaymentMethod (CreditCard, UPI, PayPal), ShippingMethod (Standard, Express).
- State: Order goes through PLACED → CONFIRMED → SHIPPED → DELIVERED.
- Composition: Cart HAS-A CartItems; Order HAS-A Items.

```
class Product {  
    private String id;  
    private String name;  
    private double price;  
    // ... getters  
}  
  
class CartItem {  
    private Product product;  
    private int quantity;  
    public double getTotal() { return product.getPrice() * quantity; }  
}  
  
class Cart {  
    private List<CartItem> items = new ArrayList<>();  
    public void add(Product p, int qty) {  
        items.add(new CartItem(p, qty));  
    }  
    public double total() {  
        return items.stream().mapToDouble(CartItem::getTotal).sum();  
    }  
}  
  
interface PaymentMethod {  
    boolean pay(double amount);  
}  
  
interface ShippingMethod {  
    double cost(double weight);  
    Duration estimatedDelivery();  
}
```

SCENARIO 8: Chess Game

Q8. Design a chess game (basic OO structure).

✓ PERFECT ANSWER

Entities: Board, Cell, Piece (abstract), King/Queen/Rook/Bishop/Knight/Pawn, Player, Game.

OOP applied:

- Inheritance: each piece type extends Piece, overriding canMove().
- Polymorphism: same canMove() signature, different rules per piece.
- Composition: Board HAS-A Cells; Cell HAS-A Piece.

```
enum Color { WHITE, BLACK }

abstract class Piece {
    protected Color color;

    public Piece(Color c) { this.color = c; }

    public abstract boolean canMove(Board b, Cell from, Cell to);
}

class King extends Piece {
    public King(Color c) { super(c); }

    public boolean canMove(Board b, Cell from, Cell to) {
        int dx = Math.abs(from.x - to.x);
        int dy = Math.abs(from.y - to.y);
        return dx <= 1 && dy <= 1 && !sameColor(to);
    }

    private boolean sameColor(Cell to) {
        return to.piece != null && to.piece.color == this.color;
    }
}

class Rook extends Piece {
    public Rook(Color c) { super(c); }

    public boolean canMove(Board b, Cell from, Cell to) {
        return (from.x == to.x || from.y == to.y) && pathClear(b, from, to);
    }

    private boolean pathClear(Board b, Cell from, Cell to) { return true; /*
check path */ }
}

class Cell {
    int x, y;
```

SCENARIO 9: File System (Composite Pattern) ★★★★★

Q9. Design a file system with files and folders.

✓ PERFECT ANSWER

Pattern: Composite — Folder contains a list of FileSystemEntry, which can be either File or Folder (recursive structure).

OOP applied:

- Abstraction: FileSystemEntry is the common interface.
- Polymorphism: getSize() works on file or folder.
- Recursion: Folder.getSize() sums child sizes.

```
abstract class FileSystemEntry {  
    protected String name;  
    protected FileSystemEntry parent;  
    public FileSystemEntry(String name) { this.name = name; }  
    public abstract long getSize();  
    public abstract void print(String indent);  
}  
  
class File extends FileSystemEntry {  
    private long size;  
    public File(String name, long size) {  
        super(name); this.size = size;  
    }  
    @Override public long getSize() { return size; }  
    @Override public void print(String i) {  
        System.out.println(i + name + " (" + size + " bytes)");  
    }  
}  
  
class Folder extends FileSystemEntry {  
    private List<FileSystemEntry> children = new ArrayList<>();  
  
    public Folder(String name) { super(name); }  
  
    public void add(FileSystemEntry e) {  
        children.add(e);  
        e.parent = this;  
    }  
  
    @Override  
    public long getSize() {
```

SCENARIO 10: Shape Hierarchy with Visitor Pattern

Q10. Design a shape hierarchy supporting multiple operations (area, render, serialize) without modifying existing classes.

✓ PERFECT ANSWER

Pattern: Visitor — separates operations from the shape classes. Add new operations without modifying shapes.

Use case: When you have a stable hierarchy but evolving operations.

```
interface ShapeVisitor<T> {

    T visit(Circle c);

    T visit(Rectangle r);

    T visit(Triangle t);

}

abstract class Shape {

    public abstract <T> T accept(ShapeVisitor<T> v);

}

class Circle extends Shape {

    double radius;

    public Circle(double r) { this.radius = r; }

    @Override public <T> T accept(ShapeVisitor<T> v) { return v.visit(this); }

}

class Rectangle extends Shape {

    double width, height;

    public Rectangle(double w, double h) { width = w; height = h; }

    @Override public <T> T accept(ShapeVisitor<T> v) { return v.visit(this); }

}

class Triangle extends Shape {

    double base, height;

    public Triangle(double b, double h) { base = b; height = h; }

    @Override public <T> T accept(ShapeVisitor<T> v) { return v.visit(this); }

}

// New operation: area – no change to shape classes
```

SCENARIO 11–15: Quick Design Problems ★★

Q11. Design a hotel reservation system — key classes.

✓ PERFECT ANSWER

- Hotel, Room (abstract), SingleRoom, DoubleRoom, Suite, Reservation, Guest, RoomService.
- Strategy: PricingStrategy varies by room type and season.
- State: Reservation status — PENDING, CONFIRMED, CHECKED_IN, CHECKED_OUT, CANCELLED.
- Composition: Hotel HAS-A Rooms; Reservation HAS-A Room + Guest.

Q12. Design an online food ordering system.

✓ PERFECT ANSWER

- Customer, Restaurant, Menu, MenuItem, Order, DeliveryAgent.
- Inheritance: User abstract → Customer, RestaurantOwner, DeliveryAgent.
- Strategy: PaymentMethod, DeliveryStrategy.
- Observer: notify Customer on order status changes.

Q13. Design a movie ticket booking system (BookMyShow).

✓ PERFECT ANSWER

- Movie, Theater, Show, Seat, Booking, Payment.
- Seat hierarchy: RegularSeat, PremiumSeat, RecliningSeat.
- State pattern: Booking states — RESERVED, PAID, CONFIRMED, CANCELLED.
- Concurrency: Seat locking during booking (handle race conditions).

Q14. Design a deck of cards for a poker/blackjack game.

✓ PERFECT ANSWER

- Card, Deck, Hand, Player, Game.
- Enums: Suit (HEARTS, DIAMONDS, CLUBS, SPADES), Rank (TWO...ACE).
- Inheritance: Game abstract → PokerGame, BlackjackGame override scoring.
- Polymorphism: each game type defines its own rules.

Q15. Design a snake-and-ladder game (or similar board game).

✓ PERFECT ANSWER

- Board, Cell, Player, Dice, Game, Snake, Ladder, Jump (abstract).
- Composition: Board HAS-A Cells; Cell HAS-A Optional<Jump>.
- Polymorphism: Snake and Ladder both implement Jump.affectPosition().
- State: Game state machine — INIT, IN_PROGRESS, ENDED.



SUMMARY: Scenario-Based Design Framework

- Always start with entities → classes.
- Identify relationships → inheritance vs composition.
- Apply OOP principles explicitly.
- Mention applicable design patterns (Strategy, State, Observer, Composite, Visitor).
- Discuss extensibility — Open/Closed principle compliance.

PART 5: CODING QUESTIONS (OOP FOCUSED)

These coding problems test your ability to APPLY OOP principles in working code. Each solution demonstrates encapsulation, inheritance, polymorphism, and design pattern usage. Practice typing these out — interviewers often ask you to code on a whiteboard or shared editor.

PROBLEM 1: Bank Account Hierarchy ★★★★★

Q1. Create a class hierarchy for bank accounts (Savings, Current, FixedDeposit) with appropriate methods, interest calculation, and withdrawal rules.

Solution Approach

✓ PERFECT ANSWER

- 149. Identify common features → abstract Account class.
- 150. Identify type-specific behavior → subclasses.
- 151. Apply encapsulation: private fields + controlled accessors.
- 152. Use polymorphism for calculateInterest() and withdraw().
- 153. Add domain validations (positive amounts, minimum balance).

Complete Implementation

```
import java.time.*;
import java.util.*;

public abstract class Account {
    private final String accountNumber;
    private final String holderName;
    protected double balance;
    private final List<Transaction> history = new ArrayList<>();

    protected Account(String accNum, String name, double opening) {
        if (opening < 0) throw new IllegalArgumentException("Negative
opening");
        this.accountNumber = accNum;
        this.holderName = name;
        this.balance = opening;
    }

    public abstract double calculateInterest();
    public abstract boolean withdraw(double amount);

    public void deposit(double amount) {
        if (amount <= 0) throw new IllegalArgumentException();
        balance += amount;
        history.add(new Transaction(TransactionType.DEPOSIT, amount));
    }

    public double getBalance() { return balance; }
    public String getAccountNumber() { return accountNumber; }
    public List<Transaction> getHistory() {
        return Collections.unmodifiableList(history);
    }
}
```



WHAT INTERVIEWER TESTS

- Demonstrates inheritance, polymorphism, and encapsulation.
- Uses final fields for immutable identity.
- Defensive copies (unmodifiableList) for collection access.
- Domain validation in setters and constructors.

PROBLEM 2: Shape Hierarchy with Polymorphism

Q2. Design a Shape hierarchy with abstract methods for area and perimeter. Demonstrate polymorphism using an array.

```
abstract class Shape {  
    private String color;  
    public Shape(String color) { this.color = color; }  
  
    public abstract double area();  
    public abstract double perimeter();  
  
    public String getColor() { return color; }  
  
    @Override  
    public String toString() {  
        return getClass().getSimpleName() + "(color=" + color +  
            ", area=" + area() + ", perimeter=" + perimeter() + ")";  
    }  
}  
  
class Circle extends Shape {  
    private final double radius;  
    public Circle(String c, double r) { super(c); this.radius = r; }  
    @Override public double area() { return Math.PI * radius *  
radius; }  
    @Override public double perimeter() { return 2 * Math.PI * radius; }  
}  
  
class Rectangle extends Shape {  
    private final double width, height;  
    public Rectangle(String c, double w, double h) {  
        super(c); this.width = w; this.height = h;  
    }  
    @Override public double area() { return width * height; }  
    @Override public double perimeter() { return 2 * (width + height); }
```

PROBLEM 3: Implement Singleton Pattern (Thread-Safe) ★★★★★

Q3. Implement the Singleton pattern in multiple ways and explain trade-offs.

```
// 1. Eager initialization – thread-safe, but instance created even if
unused

class EagerSingleton {

    private static final EagerSingleton INSTANCE = new EagerSingleton();

    private EagerSingleton() {}

    public static EagerSingleton getInstance() { return INSTANCE; }

}

// 2. Lazy with synchronized method – thread-safe but slow (every call
synchronized)

class LazySingleton {

    private static LazySingleton instance;

    private LazySingleton() {}

    public static synchronized LazySingleton getInstance() {

        if (instance == null) instance = new LazySingleton();

        return instance;

    }

}

// 3. Double-checked locking – thread-safe and fast (modern recommended)

class DCLSingleton {

    private static volatile DCLSingleton instance;

    private DCLSingleton() {}

    public static DCLSingleton getInstance() {

        if (instance == null) {                                // first check (no
lock)

            synchronized (DCLSingleton.class) {

                if (instance == null) {                        // second check
(with lock)

                    instance = new DCLSingleton();

                }

            }

        }

    }

}
```

Approach	Thread-Safe	Lazy	Performance	Recommended
Eager	✓	✗	Fast	If always used
Synchronized method	✓	✓	Slow	Avoid
Double-checked locking	✓	✓	Fast	OK
Bill Pugh Holder	✓	✓	Fast	✓ Best (for class-based)
Enum	✓	✓	Fast	✓ Best (overall)

PROBLEM 4: Implement Builder Pattern ★★★★★

Q4. Implement a Builder pattern for a User class with many optional fields.

```
public class User {

    private final String firstName;           // required
    private final String lastName;           // required
    private final int age;                    // optional
    private final String email;               // optional
    private final String phone;               // optional
    private final String address;             // optional

    private User(Builder b) {
        this.firstName = b.firstName;
        this.lastName  = b.lastName;
        this.age        = b.age;
        this.email       = b.email;
        this.phone       = b.phone;
        this.address     = b.address;
    }

    // Getters omitted for brevity

    public static class Builder {
        private final String firstName;
        private final String lastName;
        private int age;
        private String email;
        private String phone;
        private String address;

        public Builder(String firstName, String lastName) {
            this.firstName = firstName;
            this.lastName  = lastName;
        }
    }
}
```

PROBLEM 5: Implement Factory Pattern

Q5. Implement a simple Factory pattern that returns different types of Shape based on input.

```
interface Shape {  
    void draw();  
}  
  
class Circle    implements Shape { public void draw()  
{ System.out.println("Circle"); } }  
  
class Rectangle implements Shape { public void draw()  
{ System.out.println("Rectangle"); } }  
  
class Triangle  implements Shape { public void draw()  
{ System.out.println("Triangle"); } }  
  
class ShapeFactory {  
    public static Shape create(String type) {  
        if (type == null) throw new IllegalArgumentException();  
        return switch (type.toLowerCase()) {           // Java 14+ switch  
expression  
            case "circle"    -> new Circle();  
            case "rectangle" -> new Rectangle();  
            case "triangle"  -> new Triangle();  
            default -> throw new IllegalArgumentException("Unknown: " +  
type);  
        };  
    }  
}  
  
// Usage  
Shape s1 = ShapeFactory.create("circle");  
Shape s2 = ShapeFactory.create("triangle");  
s1.draw();    // Circle  
s2.draw();    // Triangle
```

PROBLEM 6: Implement Observer Pattern

Q6. Implement the Observer pattern for a stock-price notification system.

```
import java.util.*;

interface Observer {
    void update(String stock, double price);
}

interface Subject {
    void register(Observer o);
    void unregister(Observer o);
    void notifyObservers();
}

class Stock implements Subject {
    private final String symbol;
    private double price;
    private final List<Observer> observers = new ArrayList<>();

    public Stock(String symbol) { this.symbol = symbol; }

    public void setPrice(double p) {
        this.price = p;
        notifyObservers();
    }

    @Override public void register(Observer o) { observers.add(o); }
    @Override public void unregister(Observer o) { observers.remove(o); }

    @Override
    public void notifyObservers() {
        for (Observer o : observers) o.update(symbol, price);
    }
}
```

PROBLEM 7: Create a Truly Immutable Class

Q7. Create an immutable class with a mutable field (List) — handle defensive copies properly.

```
import java.util.*;

public final class ImmutableEmployee {           // 1. final class

    private final String name;                   // 2. private final fields
    private final int age;

    private final List<String> skills;           // mutable type – needs
    defensive copy

    public ImmutableEmployee(String name, int age, List<String> skills) {

        this.name = name;

        this.age = age;

        // 3. Defensive copy IN
        this.skills = new ArrayList<>(skills);
    }

    public String getName() { return name; }

    public int getAge()      { return age; }

    public List<String> getSkills() {

        // 4. Defensive copy OUT – caller cannot modify our internal state
        return new ArrayList<>(skills);

        // OR: return Collections.unmodifiableList(skills);
    }
}

// Test
List<String> skills = new ArrayList<>(Arrays.asList("Java", "Spring"));
ImmutableEmployee e = new ImmutableEmployee("Md", 21, skills);

skills.add("Hack");                               // External mutation
```

PROBLEM 8: Animal Hierarchy with Multiple Interfaces

Q8. Design an animal hierarchy where some animals can fly, some can swim, some can do both.

```
interface Swimmer {  
    default void swim() { System.out.println("Swimming"); }  
}  
  
interface Flyer {  
    default void fly() { System.out.println("Flying"); }  
}  
  
interface Walker {  
    default void walk() { System.out.println("Walking"); }  
}  
  
abstract class Animal {  
    protected String name;  
    public Animal(String name) { this.name = name; }  
    public abstract String sound();  
}  
  
class Dog extends Animal implements Walker, Swimmer {  
    public Dog(String n) { super(n); }  
    public String sound() { return "Woof"; }  
}  
  
class Fish extends Animal implements Swimmer {  
    public Fish(String n) { super(n); }  
    public String sound() { return "Blub"; }  
}  
  
class Bird extends Animal implements Flyer, Walker {  
    public Bird(String n) { super(n); }  
    public String sound() { return "Tweet"; }
```

PROBLEM 9: Implement a Generic Stack

Q9. Implement a generic Stack class with push, pop, peek, isEmpty, size.

```
public class Stack<E> {  
    private Object[] elements;  
    private int size = 0;  
    private static final int DEFAULT_CAPACITY = 16;  
  
    public Stack() {  
        elements = new Object[DEFAULT_CAPACITY];  
    }  
  
    public void push(E item) {  
        ensureCapacity();  
        elements[size++] = item;  
    }  
  
    @SuppressWarnings("unchecked")  
    public E pop() {  
        if (isEmpty()) throw new EmptyStackException();  
        E result = (E) elements[--size];  
        elements[size] = null;          // prevent memory leak  
        return result;  
    }  
  
    @SuppressWarnings("unchecked")  
    public E peek() {  
        if (isEmpty()) throw new EmptyStackException();  
        return (E) elements[size - 1];  
    }  
  
    public boolean isEmpty() { return size == 0; }  
    public int size()          { return size; }  
}
```

PROBLEM 10: Implement equals, hashCode, clone, toString

Q10. Implement a Point class with proper equals, hashCode, toString, and clone methods.

```
import java.util.Objects;

public class Point implements Cloneable {

    private double x;

    private double y;

    public Point(double x, double y) { this.x = x; this.y = y; }

    public double getX() { return x; }

    public double getY() { return y; }

    @Override

    public boolean equals(Object obj) {

        if (this == obj) return true;           // reflexive
shortcut

        if (!(obj instanceof Point)) return false; // type check

        Point other = (Point) obj;

        return Double.compare(x, other.x) == 0

            && Double.compare(y, other.y) == 0;

    }

    @Override

    public int hashCode() {

        return Objects.hash(x, y);

    }

    @Override

    public String toString() {

        return "Point(" + x + ", " + y + ")";

    }

}
```

★ VVI: THE equals/hashCode CONTRACT

- Reflexive: $x.equals(x) \rightarrow true$
- Symmetric: $x.equals(y) \Leftrightarrow y.equals(x)$
- Transitive: $x.equals(y) \wedge y.equals(z) \rightarrow x.equals(z)$
- Consistent: repeated calls give same result
- Null: $x.equals(null) \rightarrow false$
- hashCode: equal objects MUST have equal hashCodes

PROBLEM 11: Implement a Singly Linked List ★★

Q11. Implement a generic LinkedList with add, remove, get, and size.

```
public class LinkedList<E> {

    private Node<E> head;

    private int size = 0;

    private static class Node<E> {

        E data;

        Node<E> next;

        Node(E data) { this.data = data; }

    }

    public void add(E item) {

        Node<E> newNode = new Node<>(item);

        if (head == null) head = newNode;

        else {

            Node<E> cur = head;

            while (cur.next != null) cur = cur.next;

            cur.next = newNode;

        }

        size++;

    }

    public E get(int index) {

        if (index < 0 || index >= size) throw new
IndexOutOfBoundsException();

        Node<E> cur = head;

        for (int i = 0; i < index; i++) cur = cur.next;

        return cur.data;

    }

    public boolean remove(E item) {

        if (head == null) return false;
```

PROBLEM 12: Vehicle Hierarchy with Method Overriding

Q12. Create a vehicle hierarchy demonstrating method overriding and covariant return types.

```
abstract class Vehicle {

    protected String make;

    protected String model;

    public Vehicle(String make, String model) {

        this.make = make; this.model = model;

    }

    public abstract double getFuelEfficiency();           // km/L

    public abstract Vehicle createClone();               // covariant return

    public final String describe() {                     // template method

        return make + " " + model + " - " + getFuelEfficiency() + " km/L";

    }

}

class Car extends Vehicle {

    private double mileage;

    public Car(String make, String model, double mileage) {

        super(make, model); this.mileage = mileage;

    }

    @Override public double getFuelEfficiency() { return mileage; }

    @Override

    public Car createClone() {                           // covariant return

        return new Car(make, model, mileage);

    }

}

class Bike extends Vehicle {

    private double mileage;
```

PROBLEM 13: Demonstrate Composition over Inheritance

Q13. Implement an Employee class using composition (Address, Salary, Department) instead of inheritance.

```
class Address {  
    private String street, city, zip;  
    public Address(String s, String c, String z) {  
        street = s; city = c; zip = z;  
    }  
    @Override public String toString() {  
        return street + ", " + city + " " + zip;  
    }  
}  
  
class Salary {  
    private double base;  
    private double bonus;  
    public Salary(double base, double bonus) { this.base = base; this.bonus  
= bonus; }  
    public double total() { return base + bonus; }  
}  
  
class Department {  
    private String name;  
    private String head;  
    public Department(String name, String head) { this.name = name;  
this.head = head; }  
}  
  
class Employee {  
    private String id;  
    private String name;  
    private Address address;           // HAS-A  
    private Salary salary;             // HAS-A  
    private Department department;     // HAS-A
```

PROBLEM 14: Multi-Level Inheritance

Q14. Create a multi-level inheritance: Person → Student → GraduateStudent.

```
class Person {  
    protected String name;  
    protected int age;  
  
    public Person(String name, int age) {  
        this.name = name; this.age = age;  
    }  
  
    public void introduce() {  
        System.out.println("Hi, I'm " + name + ", " + age + " years old.");  
    }  
}  
  
class Student extends Person {  
    protected String university;  
    protected double gpa;  
  
    public Student(String name, int age, String uni, double gpa) {  
        super(name, age);  
        this.university = uni;  
        this.gpa = gpa;  
    }  
  
    @Override  
    public void introduce() {  
        super.introduce();  
        System.out.println("I study at " + university + " with GPA " + gpa);  
    }  
}  
  
class GraduateStudent extends Student {
```

PROBLEM 15: Sort Employees with Multiple Comparators

Q15. Demonstrate Comparable for natural ordering and Comparator for custom sorting.

```
import java.util.*;

class Employee implements Comparable<Employee> {

    String name;

    int id;

    double salary;

    LocalDate joinDate;

    public Employee(String name, int id, double salary, LocalDate joinDate)
    {
        this.name = name; this.id = id; this.salary = salary; this.joinDate
        = joinDate;
    }

    @Override

    public int compareTo(Employee o) {

        return Integer.compare(this.id, o.id);           // Natural order by
id

    }

    public String getName()          { return name; }

    public double getSalary()        { return salary; }

    public LocalDate getJoinDate()   { return joinDate; }

    @Override

    public String toString() { return name + "(" + id + "," + salary +
    ")"; }

}

List<Employee> employees = new ArrayList<>(List.of(

    new Employee("Alice", 3, 90000, LocalDate.of(2020, 1, 5)),

    new Employee("Bob",    1, 75000, LocalDate.of(2021, 6, 12)),
```



SUMMARY: OOP Coding Problems

- Always identify the right abstraction (interface or abstract class).
- Apply encapsulation: private fields, validated mutators.
- Use polymorphism for behavior that varies by type.
- Prefer composition for assembly; inheritance for genuine IS-A.
- Master Singleton, Builder, Factory, Observer, Strategy patterns.

PART 6: RAPID FIRE ROUND (50 Quick Answers)

Rapid-fire questions are common during HR rounds and as warm-ups in technical rounds. Memorize these one-liners — interviewers expect instant answers without hesitation.



VVI: HOW TO USE THIS SECTION

- Cover the answer column. Read the question. Speak the answer in under 5 seconds.
- If you hesitate or are wrong, mark with **✗** and revisit later.
- Target: 95%+ correct on second pass.
- Drill this section the day before your interview.

GROUP 1: Override, Inheritance, Static (Q1–Q15)

Question	Answer
Q1. Can we override a private method?	No — private methods are not visible to subclasses.
Q2. Can we override a static method?	No — static methods are HIDDEN, not overridden (compile-time binding).
Q3. Can we override a final method?	No — final prevents overriding.
Q4. Can a constructor be final?	No — constructors cannot be inherited, so final is meaningless.
Q5. Can a constructor be static?	No — constructors initialize instance state.
Q6. Can a constructor be abstract?	No — abstract has no body; constructor must initialize.
Q7. Can a class be both abstract and final?	No — final prevents subclassing; abstract requires it.
Q8. Can a method be both abstract and final?	No — contradictory.
Q9. Can a method be both abstract and static?	No — static methods cannot be overridden.
Q10. Can a method be both abstract and private?	No — private methods cannot be overridden.
Q11. Can a constructor be inherited?	No — constructors are not inherited.
Q12. Does Java support multiple inheritance?	Not through classes; YES through interfaces.
Q13. Can we override the main method?	No — main is static (can be overloaded).
Q14. Can main method be overloaded?	Yes — but JVM calls only the standard signature.
Q15. Can main method be final?	Yes — though no practical effect (static methods aren't overridden anyway).

GROUP 2: Interface & Abstract Class (Q16–Q30)

Question	Answer
Q16. Can an interface extend a class?	No — only other interfaces.

Question	Answer
Q17. Can an interface extend multiple interfaces?	Yes.
Q18. Can a class implement multiple interfaces?	Yes — Java's form of multiple inheritance.
Q19. Can an abstract class have a constructor?	Yes — invoked from subclass via super().
Q20. Can an abstract class be instantiated?	No — never directly.
Q21. Can an interface have a constructor?	No — never.
Q22. Can an interface have static methods?	Yes, since Java 8.
Q23. Can an interface have default methods?	Yes, since Java 8.
Q24. Can an interface have private methods?	Yes, since Java 9.
Q25. Can an interface have instance variables?	No — only public static final constants.
Q26. Default modifier for interface methods?	public abstract (implicit).
Q27. Default modifier for interface fields?	public static final (implicit).
Q28. Can an abstract class have no abstract methods?	Yes — but it still cannot be instantiated.
Q29. Can an abstract class have main method?	Yes — main is static.
Q30. Can an interface have main method?	Yes, since Java 8 (static method).

GROUP 3: Objects, this, super, Misc (Q31–Q40)

Question	Answer
Q31. Can 'this' be used in a static method?	No — no instance context.
Q32. Can 'super' be used in a static method?	No — same reason.
Q33. Can a class extend itself?	No — circular inheritance.
Q34. Can a static method access non-static members?	Not directly — only via an instance reference.
Q35. Where are objects stored in memory?	Heap (references on stack).
Q36. Default value of an instance int field?	0.
Q37. Default value of an instance reference field?	null.
Q38. Default value of a local variable?	None — compile error if used before init.
Q39. Order: static block, instance block, constructor?	Static → main → instance block → constructor.
Q40. What is the parent class of all classes?	java.lang.Object.

GROUP 4: Polymorphism, OOP Pillars (Q41–Q50)

Question	Answer
Q41. What is method overloading?	Same name, different parameters in same class.
Q42. What is method overriding?	Child redefines parent's method with same signature.
Q43. Compile-time or runtime: overloading?	Compile-time (static binding).
Q44. Compile-time or runtime: overriding?	Runtime (dynamic binding).
Q45. Are fields polymorphic in Java?	No — only methods. Fields use reference type.
Q46. What is covariant return type?	Override may return a subtype of parent's return type.
Q47. What is upcasting?	Subclass reference → superclass type (implicit, safe).
Q48. What is downcasting?	Superclass reference → subclass type (explicit, risky).
Q49. Why override hashCode when equals is overridden?	Contract: equal objects MUST have same hashCode (HashMap/HashSet depend on it).
Q50. Four pillars of OOP?	Encapsulation, Inheritance, Polymorphism, Abstraction.



SUMMARY: Rapid Fire Mastery Checklist

- Cover-and-recall: practice without looking at answers.
- Target speed: under 5 seconds per question.
- After mastering, randomize the order to avoid pattern memorization.
- Drill the day before interview — confidence boost.
- If you miss a question, write it on a flashcard and re-quiz.

PART 7: VVI INTERVIEW TOPICS (Priority List)

This section ranks topics by their **FREQUENCY** in real Java backend interviews. Revise the ★★★★★ items the night before; they appear in 90%+ of interviews.

★★★★★ MUST KNOW (Asked in 90%+ Interviews)

★ VVI: THESE TOPICS ARE NON-NEGOTIABLE

If you fail on any of these, the interview likely ends. Spend at least 60% of revision time here.

Topic	Why Asked	Where in this Bank
Polymorphism (overload + override)	Tests fundamental OOP understanding	Part 2 → Topic 2.2
Interface vs Abstract Class	Tests design judgment	Part 3 → Topic 3.3
Inheritance types + diamond problem	Tests Java specifics	Part 2 → Topic 2.1
Encapsulation with example	Tests data-hiding understanding	Part 2 → Topic 2.3
Constructor types & chaining	Tests JVM init knowledge	Part 1 → Topic 1.2
Access modifiers (private/public/protected/default)	Tests visibility scope	Part 2 → Topic 2.5
this vs super	Tests reference semantics	Part 1 → Topic 1.3 + 2.1
equals() & hashCode() contract	Tests Collections framework prep	Part 5 → Problem 10
Method overriding rules	Tests inheritance fluency	Part 2 → Topic 2.1
Singleton pattern (thread-safe)	Tests design patterns + concurrency	Part 5 → Problem 3

★★★★★ VERY IMPORTANT (Asked in 70%+ Interviews)

Topic	Typical Question Style	Where
Composition vs Inheritance	When would you choose composition?	Part 3 → Topic 3.4
Overloading vs Overriding (table)	Tabular comparison expected	Part 2 → Topic 2.2
Abstract class — when to use	Real scenario design	Part 3 → Topic 3.2

Topic	Typical Question Style	Where
Interface (Java 8+ default methods)	Knowledge of language evolution	Part 3 → Topic 3.1
Immutable class — how to create	Code on whiteboard	Part 5 → Problem 7
static keyword (variable/method/block)	JVM execution order question	Part 1 → Topic 1.4
Marker interface, Functional interface	Java 8 prep	Part 3 → Topic 3.1
Builder pattern	Code or design discussion	Part 5 → Problem 4
Encapsulation vs Abstraction	Side-by-side comparison	Part 2 → Topic 2.4
Stack vs Heap memory	JVM internals	Part 1 → Topic 1.1

☆☆☆ IMPORTANT (Asked in 50%+ Interviews)

Topic	Typical Question Style	Where
Coupling and Cohesion	Definition + impact on design	Part 3 → Topic 3.6
Aggregation vs Composition	Lifecycle distinction	Part 3 → Topic 3.5
SOLID principles (intro)	Brief explanation each	Part 3 → Topic 3.6
Factory pattern, Observer pattern	Implement on whiteboard	Part 5 → Problems 5, 6
Comparable vs Comparator	Real example expected	Part 3 → Topic 3.1
Anonymous & inner classes	When/why to use	Part 1 → Topic 1.1 + general
Final keyword (variable/method/class)	Each context	Part 1 + Part 2
Garbage collection basics	What is GC-eligible?	Part 1 → Topic 1.1
instanceof, casting	Pattern matching (Java 16+)	Part 2 → Topic 2.2
Object class methods	List 9 inherited methods	Part 1 → Topic 1.1



7-DAY INTERVIEW PREP CALENDAR

Day 7 (1 week before)

- Read Parts 1–3 (Basic + Intermediate + Advanced).
- Cover answers, attempt every question.
- Mark weak spots with a highlighter.

Day 6

- Review weak spots from yesterday.
- Read Part 4 (Scenarios) — speak design out loud.
- Practice ATM, Library, Parking Lot designs.

Day 5

- Code Part 5 problems on paper/whiteboard.
- Focus: Bank Account hierarchy, Singleton, Builder.
- Time yourself: under 15 minutes per problem.

Day 4

- Drill Part 6 (Rapid Fire) — aim for 95% in under 5 sec each.
- Re-revise Interface vs Abstract Class section.
- Practice explaining encapsulation in 30 seconds out loud.

Day 3

- Mock interview — solo speaking practice.
- Pick 5 random questions from each part. Time: 30 seconds each.
- Record yourself; review tone and clarity.

Day 2

- Final pass on ★★★★★ 5-star topics only.
- Re-read summary boxes in Parts 1–3.
- Rest well; do not learn new material.

Day 1 (Interview Day)

- Light revision: Rapid Fire (Part 6) + Decision Framework for Interface vs Abstract.
- Have one favorite real-world scenario ready (e.g., parking lot, library).
- Be confident; stay calm. You've prepared thoroughly.



TOP 10 ANSWERS TO MEMORIZE WORD-FOR-WORD

1. What is encapsulation?

"Encapsulation is the bundling of data and the methods that operate on that data into a single unit, and restricting direct access to the internal state. In Java, we achieve it by declaring fields as private and providing public getters and setters with validation."

2. What is polymorphism?

"Polymorphism means many forms — the same entity can behave differently in different contexts. Java has two types: compile-time polymorphism through method overloading, where same method name has different parameters, and runtime polymorphism through method overriding, where child class redefines a parent method, resolved at runtime via dynamic dispatch."

3. Interface vs Abstract Class?

"Use an interface when defining a contract for unrelated classes that share a capability, like Comparable or Serializable. Use an abstract class when related classes share substantial common code and you need state, constructors, or non-public members. Interface is CAN-DO; abstract class is IS-A. In modern Java, I prefer interfaces with default methods unless I need abstract class features."

4. Why is String immutable?

"For five reasons: string pool optimization to share instances safely, thread safety without synchronization, security since strings are used in class names and URLs, hashCode caching for performance, and ClassLoader integrity."

5. Why doesn't Java support multiple inheritance through classes?

"To avoid the diamond problem — if two parent classes have a method with the same signature, the compiler can't decide which to inherit. Java solves it by allowing only single inheritance through classes, while supporting multiple inheritance through interfaces, with explicit resolution rules for conflicting default methods."

6. Composition vs Inheritance?

"Favor composition over inheritance. Composition is HAS-A — flexibility, loose coupling, runtime behavior change via dependency injection. Inheritance is IS-A — use it only when there's a true type hierarchy and the parent is designed for extension. Composition avoids the fragile base class problem and the diamond problem."

7. What is the equals and hashCode contract?

"If two objects are equal via equals(), they must return the same hashCode. The contract is reflexive, symmetric, transitive, and consistent. Violating it breaks hash-based collections like HashMap and HashSet — objects get lost in buckets."

8. How do you create an immutable class?

"Five rules: declare the class final, make all fields private and final, no setters, initialize all fields via constructor, and for mutable field types, make defensive copies both incoming via constructor and outgoing via getters."

9. Can we override static methods?

"No, static methods cannot be overridden. They are resolved at compile time based on reference type, not runtime object type. If a subclass declares a static method with the same signature, it's method Hiding, not overriding. Polymorphism doesn't apply to static methods."

10. What is the difference between abstraction and encapsulation?

"Abstraction hides HOW something is done — focuses on essential behavior, achieved through abstract classes and interfaces. Encapsulation hides WHAT data is stored — focuses on data protection, achieved through private fields with controlled access. Abstraction is design-level; encapsulation is implementation-level."

FINAL THOUGHTS

Remember:

"Interviewers are not testing if you remember every detail. They're testing if you can think clearly about design, communicate trade-offs, and write maintainable code."

Three things to always do in an OOP interview:

- 154. Think out loud — narrate your reasoning, not just the answer.
- 155. Pair every concept with a real-world analogy AND a code example.
- 156. When in doubt about design, choose composition + interface — modern Java way.

Career Path Reminder

This question bank covers ONE pillar of your Java Backend Developer journey:

- ✓ Core Java + OOP (this bank)
- → Java 8 Features (Streams, Lambdas, Optional)
- → DSA (NeetCode 150, pattern-based)
- → Advanced Java (Collections, Multithreading, Exceptions)
- → Spring Core → Spring Boot → REST APIs → Microservices
- → Database (MySQL/PostgreSQL), Git, Docker, AWS

Next milestone: Master Java 8 features (Streams API, Lambdas, Functional Interfaces) — they appear in every modern interview.

END OF QUESTION BANK

Total: 100+ questions across 7 parts.

You now have a comprehensive reference for OOP interviews.

Practice, revise, and walk into your interview with confidence.

All the best for your Java Backend Developer journey! 🚀